Predictive Modelling of Social Media Trend Emergence: A Reddit-Based Study

Student Name: Jolin Faith lim

Course code: CM3070

Date: 23/03/2025

Templete: 1.2 Project Idea 2: Predictive Modelling of Social Media Trend Emergence

Code Repository: https://github.com/oclockwise/Predictive-Modelling-of-Social-Media-Trend-Emergence-A-Reddit-Based-Study.git

Contents

1 INTRODUCTION (920 words) 5
1.1 Project Concept and Motivation 5
1.2 Project Idea and Template Alignment 6
1.3 Aims and Research Questions 7
1.4 Project Deliverables and Objectives 7
2 LITERATURE REVIEW (2490 words) 8
2.1 Factors Influencing Social Media Virality 8
2.1.1 Emotional Expression and Engagement 8
2.1.2 Temporal Dynamics and Early Visibility 9
2.1.3 Community Structure and Reddit Virality 9
2.2 Machine-Learning Approaches to Predicting Virality 9
2.2.1 Linear Models: Logistic Regression and Linear Regression 10
2.2.2 Ensemble Models: Random Forest and XGBoost 10
2.2.3 Synthesis Across Modelling Approaches 11
2.3 Sentiment Analysis and Emotional Influence 11
2.3.1 How Emotion Shapes Online Engagement 11
2.3.2 Rationale for Using VADER 11
2.3.3 Limitations of Sentiment Modelling 12
2.4 Time-Series Dynamics in Virality Growth 12
2.4.1 Temporal Patterns and Visibility 12
2.4.2 ARIMA and Short-Term Forecasting 13
2.5 Network Diffusion and Community Structures 13
2.5.1 Core Concepts 13
2.5.2 Reddit's Community-Based Diffusion 14
2.5.3 Incorporating Network-Inspired Features 14
2.6 Tools Used in Prior Research 14
2.7 Gaps in Existing Research and Study Contribution 14
3 PROJECT DESIGN (1996 words) 15
3.1 Domain and Users 15
3.2 User Needs and Design Requirements 16
3.2.1 Clear organisation of data 16
3.2.2 Interpretability 16

3.2.3 Support for multiple perspectives 16
3.2.4 Visual exploration 16
3.2.5 Reproducible workflow 16
3.3 Overall System Structure 17
3.3.1 Data Gathering Approach 17
3.3.2 Data Preparation and Feature Engineering 17
3.3.3 Predictive Modelling Framework 18
3.3.4 Time-Series Component 19
3.3.5 Network Analysis Component 19
3.3.6 Prototype Design 19
3.4 Technologies and Tools 20
3.5 Project Development Plan 21
3.6 Evaluation Strategy 21
3.6.1 Evaluation for Classification Models 22
3.6.2 Evaluation for Regression Models 22
3.6.3 Evaluation of Time-Series Analysis 22
3.6.4 Evaluation of Network Analysis 22
3.7 Data Constraints and Their Implications 23
4 IMPLEMENTATION (2213 words) 23
4.1 Dataset Preparation and Cleaning 23
4.2 Text Processing and Feature Engineering 27
4.2.1 Text Preprocessing 27
4.2.2 Feature Engineering 28
4.3 Text Vectorisation and Data Preparation for Modelling 29
4.3.1 Train–Test Split 29
4.3.2 Community-level Feature Construction and Text Vectorisation using TF-IDF 30
4.3.4 Feature Scaling and Final Feature Matrix 31
4.4 Machine Learning Model Implementation 32
4.4.1 Classification Models (RQ1) 32
4.4.2 Regression Models (RQ2) 38
4.4.3 Time-Series Analysis of Reddit Engagement (RQ3) 42
4.4.4 Subreddit Community Network Analysis (RQ4) 47
5 EVALUATION (2526 words) 54

5.1 Evaluation of Viral Post Classification (RQ1) 54
5.1.1 Classification Performance Comparison 54
5.1.2 ROC and Precision–Recall Analysis 56
5.1.4 Confusion Matrix Interpretation 57
5.1.5 Cross-Validation Stability 58
5.2 Evaluation of Upvote Prediction (RQ2) 58
5.2.1 Model Comparison 58
5.2.2 Actual vs Predicted Scatter Plot 59
5.2.3 Residual Plot Comparison 60
5.2.4 Additional Model Diagnostic Analysis 61
5.3 Time-Series Analysis of Reddit Engagement (RQ3) 61
5.3.1 Forecast Accuracy 61
5.3.2 Forecast vs Actual Comparison 62
5.3.3 ARIMA Forecast Behaviour 63
5.4 Evaluation of Subreddit Community Network Analysis(RQ4) 63
5.4.1 Subreddit Similarity Network Structure 63
5.4.2 Centrality and Viral Engagement 64
5.5 Limitations and Potential Improvements 66
5.6 Summary of Evaluation 67
6 CONCLUSION (286 WORDS) 68
7 APPENDIX 68
8 REFERENCES 89

1 INTRODUCTION (920 words)

Social media has become a central part of everyday communication, with platforms such as Reddit providing spaces where users share ideas, respond to events, and participate in large-scale discussions. Among the thousands of posts uploaded every day, only a small proportion attract substantial attention. Understanding why certain posts become highly visible while others receive little engagement has therefore become an important topic in digital media studies, data science, and online behaviour research.

Predicting virality is challenging because it is influenced by multiple interacting factors. The content of a post, the timing of publication, the emotional tone of the text, and the behaviour of the surrounding community can all influence how widely a post spreads. These factors rarely operate independently. Instead, they interact in complex ways that make virality difficult to explain using simple or single-variable approaches.

In this project, we analyse Reddit virality using a multi-method, data-driven framework. Reddit is particularly suitable for studying viral behaviour because the platform relies heavily on community voting rather than follower-based visibility. Posts gain prominence through upvotes and discussion rather than through influencer networks. Subreddits vary significantly in size, activity levels, moderation styles, and cultural norms, meaning that the same type of post may perform differently across communities. This environment provides a useful setting for examining how content characteristics, timing signals, and community engagement patterns contribute to viral outcomes.

The project follows Project Template 1.2: Predictive Modelling of Social Media Trend Emergence, which focuses on understanding how trends develop and how computational models can analyse these patterns.

1.1 Project Concept and Motivation

Virality is important not only for predicting engagement but also for understanding how information spreads within online communities. Highly visible posts can amplify viewpoints, attract widespread discussion, or shape conversations within a subreddit. On Reddit, where users determine which posts gain visibility through voting and commenting, viral content can briefly dominate the attention of an entire community.

There are several motivations for studying this phenomenon.

First, content creators may benefit from understanding how visibility emerges on platforms driven by community engagement. Identifying patterns associated with successful posts can help creators understand how audiences respond to different types of content.

Second, researchers treat virality as a way to study decentralised information diffusion. Unlike traditional broadcast systems, social media platforms distribute attention through collective user behaviour, providing insight into how information spreads within large digital communities.

Third, moderators and platform designers may benefit from understanding how certain posts attract unusually high levels of attention or discussion. Insights into engagement patterns can inform moderation strategies and platform design decisions.

Reddit's structure makes these motivations particularly relevant. Because subreddits operate as semi-independent communities, a post's visibility depends heavily on its specific audience. Two posts with similar content may receive very different responses depending on the subreddit in which they appear. This project therefore examines virality by combining content features, temporal signals, sentiment indicators, and subreddit-level activity within a unified analytical framework.

Although existing studies examine some of these factors individually, many focus on a single perspective such as sentiment, content structure, or network diffusion. This project addresses that limitation by integrating multiple analytical perspectives within one modelling pipeline.

1.2 Project Idea and Template Alignment

The central idea of this project is to develop a predictive modelling framework that analyses Reddit virality using several complementary analytical methods.

For the classification task, posts are labelled as viral or non-viral based on an engagement threshold derived from the distribution of Reddit upvotes within the dataset. This creates a binary prediction task in which machine-learning models attempt to distinguish high-engagement posts from typical posts.

For the regression task, the raw upvote count is used as a continuous target variable. Instead of simply identifying whether a post becomes viral, regression models estimate the magnitude of engagement a post may receive. This allows the project to explore both the presence and scale of viral behaviour.

To support these modelling tasks, several categories of features are extracted from the dataset. These include:

- textual features derived from post titles and content using TF-IDF vectorisation
- sentiment scores calculated from Reddit text
- temporal indicators such as posting hour and posting day
- subreddit-level activity measures including comment counts and posting frequency.

By combining these feature types, the project aligns closely with the goals of Template 1.2, which emphasises analysing trend formation using multiple analytical approaches. The integration of classification, regression, sentiment analysis, time-series observation, and network analysis of subreddit engagement patterns reflects the complex nature of social media virality.

1.3 Aims and Research Questions

The overall aim of this project is to develop a predictive modelling system that analyses and predicts the virality of Reddit posts using machine-learning methods and multiple feature categories.

The project is guided by the following research questions:

- **RQ1: Classification**
  - To what extent can machine-learning models distinguish viral from non-viral Reddit posts using textual, temporal, sentiment, and subreddit-level features?
- **RQ2: Regression**
  - To what extent can regression models accurately predict the eventual upvote count of Reddit posts?
- **RQ3: Time-Series Dynamics**
  - How do Reddit engagement levels evolve over time, and to what extent can time-series models capture and forecast these patterns?
- **RQ4: Network Analysis of Community Engagement**
  - How do subreddit community structure and network position influence the likelihood of Reddit posts becoming viral?

These research questions allow the project to explore both predictive performance and the behavioural patterns underlying viral engagement.

1.4 Project Deliverables and Objectives

This project aims to create a complete and practical workflow for understanding how Reddit posts gain visibility and why some become viral. The work begins by gathering data from a publicly available Reddit dataset on Kaggle.

Using this dataset, classification models distinguish viral posts from non-viral ones, while regression models estimate the expected number of upvotes. Model performance is evaluated using standard metrics such as accuracy, precision, recall, RMSE, MAE, and R^2.

In addition to predictive modelling, the project incorporates a small time-series observation to examine posting behaviour over time. The analysis also examines how network analysis of subreddit communities relates to engagement patterns and viral outcomes.

All results, visualisations, and interpretations are presented in a clear written report. Together, these deliverables provide a complete pipeline from data preparation to final interpretation, offering a well-rounded understanding of how trends emerge on Reddit and what factors contribute to viral outcomes.

2 LITERATURE REVIEW (2490 words)

This chapter examines key academic research on how social media virality is understood and predicted. Studies across TikTok, Instagram, Twitter, and Reddit consistently show that visibility is shaped not by a single factor, but by the interaction between emotional expression, temporal dynamics, community behaviour, and platform mechanisms. By synthesising these strands of research, this chapter establishes a conceptual foundation for the multi-method modelling approach adopted in this project. Rather than treating predictors in isolation, the literature suggests that virality emerges from the combined influence of content characteristics and structural platform conditions.

2.1 Factors Influencing Social Media Virality

Virality is a multi-dimensional phenomenon shaped by the interaction of content features, user behaviour, and algorithmic systems. Although platforms differ in design and recommendation logic, research consistently identifies emotional intensity, timing, and community structure as recurring drivers of visibility. Importantly, these elements do not operate independently. Engagement outcomes depend on how they interact within specific platform environments

2.1.1 Emotional Expression and Engagement

Emotional expression is one of the most consistently observed predictors of engagement. Kim and Hwang (2025) demonstrate that emotional intensity, particularly high-arousal emotions such as excitement, awe, or anger, significantly increases reaction counts. Their findings suggest that emotional activation encourages faster interaction, which can generate early engagement momentum. In this sense, emotional content functions less as a guarantee of virality and more as a catalyst for rapid audience response.

Similar conclusions are drawn by Roy and Basu (2020), who show that emotionally expressive tweets spread more quickly in political discussions. Notably, both positive and negative high-arousal emotions can stimulate sharing behaviour. This distinction is important: the literature indicates that emotional activation often matters more than simple polarity (positive versus negative tone).

However, emotional intensity alone does not fully explain visibility. Studies across platforms suggest that emotional signals amplify engagement when combined with favourable timing or

community exposure. This reinforces the need to treat sentiment as one component within a broader predictive framework rather than as a standalone driver.

2.1.2 Temporal Dynamics and Early Visibility

Timing plays a significant role in shaping early exposure. Zhang et al. (2022) and Alonso-López et al. (2020) find that content posted during peak activity windows tends to gain traction more rapidly. Posting hour, weekday cycles, and platform activity rhythms influence whether content receives initial attention.

At the same time, the literature cautions against viewing timing as deterministic. Content posted during optimal periods still depends on user response patterns and algorithmic amplification. Temporal variables therefore function as contextual modifiers: they shape the opportunity for visibility, but do not guarantee sustained engagement. This suggests that predictive models should incorporate time-based features alongside content and behavioural indicators.

2.1.3 Community Structure and Reddit Virality

Reddit presents a structurally distinct case because visibility is community-driven rather than follower-driven. Makow et al. (2021) show that subreddit size, participation rates, and voting patterns strongly influence post trajectories. In large or highly active communities, exposure opportunities increase, but competition for attention also intensifies.

Importantly, early voting behaviour can disproportionately shape outcomes. A small number of highly active users often influence initial ranking, introducing an element of unpredictability. This highlights a structural dimension of virality: success depends not only on content quality, but also on the characteristics and interaction norms of the hosting community.

Taken together, the literature suggests that virality should be understood as the product of interacting emotional, temporal, and structural factors. This integrated perspective directly informs the modelling choices made in this project, which combine sentiment, timing, and community-level features rather than relying on a single explanatory dimension.

2.2 Machine-Learning Approaches to Predicting Virality

Machine-learning models are widely used to classify content as viral or non-viral and estimate engagement. Across platforms such as TikTok, Instagram, Twitter, and Reddit, research shows predictive performance depends not only on the algorithm but also on how effectively models capture emotional, temporal, and community-level signals. Rather than identifying a universally superior model, studies highlight a trade-off between interpretability and predictive complexity. Textual information is commonly converted into numerical features using vectorisation techniques such as Term Frequency–Inverse Document

Frequency (TF-IDF), enabling machine-learning models to analyse large volumes of social media text.

2.2.1 Linear Models: Logistic Regression and Linear Regression

Linear models such as Logistic Regression and Linear Regression are frequently adopted as baseline approaches due to their simplicity and interpretability. Zhang et al. (2022) applied Logistic Regression in TikTok virality classification to assess the contribution of features including video length, posting time, and engagement velocity. Similarly, Kim and Hwang (2025) used regression analysis to examine how emotional tone relates to reaction counts.

A key strength of linear models lies in their transparency. Coefficient estimates allow researchers to determine whether variables such as sentiment intensity or subreddit characteristics exert positive or negative effects on engagement outcomes. This makes linear models valuable for explanatory analysis and theory testing.

However, multiple studies report that linear approaches are consistently outperformed by more flexible algorithms (Alonso-López et al., 2020; Zhang et al., 2022). Because linear models assume additive and independent relationships between variables, they struggle to represent the complex interactions typical of social media environments. For example, emotional intensity may increase engagement only within certain communities or during specific time windows. Such conditional relationships are difficult to capture using strictly linear assumptions. As a result, while linear models provide interpretability and serve as useful baselines, they are often insufficient as standalone predictors of virality.

2.2.2 Ensemble Models: Random Forest and XGBoost

Ensemble methods such as Random Forest and XGBoost are widely adopted in virality prediction because they can model non-linear relationships and handle high-dimensional feature spaces. Alonso-López et al. (2020) demonstrate that Random Forest achieved stronger predictive performance than Logistic Regression in Instagram popularity modelling, particularly when interaction-heavy variables were included. Zhang et al. (2022) report similar findings in TikTok studies, where tree-based models captured complex engagement dynamics more effectively than linear approaches.

Random Forest improves performance by aggregating multiple decision trees, reducing variance while preserving flexibility. XGBoost further enhances predictive accuracy through sequential boosting, correcting previous errors and capturing subtle interactions between emotional tone, posting behaviour, and community-level factors. Across studies, boosting methods frequently achieve the highest classification accuracy.

Nevertheless, this increased predictive strength introduces trade-offs. Ensemble models are less interpretable than linear models, making it harder to explain why specific features drive outcomes. They are also more computationally intensive and may overfit if hyperparameters

are not carefully tuned. The literature therefore suggests that higher accuracy does not automatically translate into better theoretical insight.

2.2.3 Synthesis Across Modelling Approaches

Across the reviewed studies, a consistent pattern emerges: linear models contribute interpretability and theoretical clarity, while ensemble and boosting methods provide superior predictive performance by capturing complex feature interactions. Importantly, no single model fully addresses both explanation and prediction.

This synthesis supports a multi-model strategy rather than reliance on one technique. By comparing linear and ensemble approaches within the same framework, researchers can evaluate both explanatory influence and predictive strength. This integrated perspective aligns with recent recommendations in social media analytics, which emphasise modelling virality as a multi-dimensional phenomenon rather than a purely statistical classification problem.

2.3 Sentiment Analysis and Emotional Influence

Sentiment analysis plays a significant role in social media research because emotional cues strongly influence how users interpret and react to content. Across platforms, studies consistently show that emotionally intense posts attract faster attention and higher engagement. However, the literature suggests that emotion does not directly cause virality, rather, it increases the likelihood of early interaction, which can subsequently trigger algorithmic amplification.

2.3.1 How Emotion Shapes Online Engagement

Kim and Hwang (2025) demonstrate that emotional intensity explains engagement patterns more effectively than simple positive-negative polarity. Their findings show that high-arousal emotions such as excitement, anger, or awe produce stronger user reactions regardless of sentiment direction. This suggests that activation level, rather than positivity alone, drives engagement. Similarly, Roy and Basu (2020) found that emotionally expressive tweets spread more rapidly because emotional cues signal urgency and relevance, prompting users to respond quickly.

Taken together, these studies indicate that emotion functions as an engagement amplifier. Emotional content increases the probability of interaction, particularly in the early stages of exposure. For Reddit, where posts frequently contain informal language, personal narratives, and expressive phrasing, sentiment scoring provides a meaningful proxy for emotional activation and potential engagement momentum.

2.3.2 Rationale for Using VADER

Among commonly used sentiment tools, VADER is particularly appropriate for this study. Unlike traditional lexicon-based tools such as TextBlob, VADER is specifically designed for short, informal social media text. It accounts for emojis, punctuation emphasis (e.g., exclamation marks), capitalisation, and slang, all of which are common in Reddit posts. This makes it better suited to capturing the expressive style typical of subreddit discussions.

In addition, VADER provides a compound sentiment score that reflects overall emotional intensity, aligning directly with literature emphasising arousal rather than simple polarity. Compared with transformer-based models, VADER is computationally efficient, interpretable, and practical for large-scale datasets, making it suitable for a multi-method framework where sentiment is one component among several predictive features.

2.3.3 Limitations of Sentiment Modelling

Despite its usefulness, sentiment analysis has limitations. Sarcasm, humour, and community-specific language reduce accuracy, and negative emotions may still generate high engagement. Therefore, sentiment should not be treated as a standalone predictor but integrated with temporal, behavioural, and community-level variables. Within this project, sentiment operates as one explanatory signal within a broader, multi-factor modelling approach.

2.4 Time-Series Dynamics in Virality Growth

Time-series analysis provides an important perspective in virality research because online visibility is inherently dynamic rather than static. Engagement unfolds over time, often accelerating rapidly in early stages before stabilising or declining. As a result, understanding when engagement occurs can be as important as understanding what content contains. Temporal rhythms such as daily posting cycles, weekly activity patterns, and short-term engagement spikes help explain how visibility emerges within platform ecosystems shaped by algorithmic ranking and user behaviour.

Unlike purely cross-sectional models, time-series approaches capture the sequential nature of interaction, allowing researchers to examine growth trajectories rather than single outcomes. This distinction is crucial, as virality is often characterised by early momentum rather than long-term accumulation.

2.4.1 Temporal Patterns and Visibility

Empirical research consistently shows that timing influences exposure and engagement (Zhang et al., 2022). Platforms exhibit predictable activity rhythms, including peak usage hours and weekly cycles, which affect how quickly posts receive attention. Posts published during high-activity windows may benefit from larger immediate audiences, whereas posts shared during quieter periods may face less competition for visibility.

Although posting time affects early exposure, Pinto et al. (2013) demonstrate that rapid initial engagement is a stronger predictor of sustained popularity than posting time alone.This suggests that temporal variables interact with behavioural signals rather than operating independently. Therefore, time should be understood as a contextual amplifier that influences exposure conditions rather than a standalone causal factor.

2.4.2 ARIMA and Short-Term Forecasting

ARIMA models are widely used to analyse short-term temporal dependencies and identify underlying trends in sequential data. In social media contexts, ARIMA has been applied to examine fluctuations in political communication and engagement patterns (Roy & Basu, 2020). These studies show that online activity often follows measurable temporal structures, including autocorrelation and seasonal variation.

While ARIMA provides interpretability and clear statistical structure, it assumes relatively stable patterns and may struggle with sudden viral spikes driven by unpredictable user behaviour. Consequently, time-series models are most valuable when combined with feature-based machine-learning approaches. Within this project, temporal analysis complements classification and regression models by revealing behavioural rhythms that shape engagement growth. Temporal variables such as posting hour, weekday patterns, and weekend indicators are incorporated as predictive features, while ARIMA modelling is used to analyse broader engagement trends over time, thereby strengthening the multi-method framework.

2.5 Network Diffusion and Community Structures

Network analysis provides a structural perspective on virality by examining how communities and posts are interconnected within a platform. Rather than focusing only on content characteristics, this approach considers how visibility emerges from patterns of interaction and relationships between different communities. In social media research, diffusion is rarely random. Instead, it reflects underlying network structures that influence how information spreads and how attention is distributed across communities. Understanding these structures helps explain why certain content gains visibility while other posts receive limited engagement.

2.5.1 Core Concepts

Network-based studies commonly analyse structural metrics such as degree centrality, betweenness centrality, and weighted node strength to understand information diffusion. Degree centrality measures the number of connections a node has within a network, indicating how broadly it is linked to other communities. Betweenness centrality captures the extent to which a node acts as a bridge between different parts of the network, potentially influencing how information flows across communities. Weighted node strength reflects the total strength of connections associated with a node, incorporating both the number and intensity of relationships. These measures provide insight into the structural position and

influence of nodes within a network and are commonly used to analyse diffusion processes in online platforms.

2.5.2 Reddit's Community-Based Diffusion

Makow et al. (2021) show that Reddit's architecture produces diffusion patterns that differ from follower-based platforms. Because Reddit is organised around topic-driven subreddits rather than personal networks, visibility is shaped largely by community participation and voting behaviour. This structure makes subreddit-level dynamics particularly important for understanding how content spreads and gains attention.

2.5.3 Incorporating Network-Inspired Features

Network-informed approaches can be used to analyse relationships between online communities by representing them as nodes within a network structure. In such networks, connections may reflect similarities in the content shared across communities, allowing researchers to examine how different groups are positioned relative to one another. Structural metrics such as degree centrality, betweenness centrality, and weighted node strength are commonly used to measure connectivity and influence within networks (Freeman, 1979). These indicators enable community structure to be examined alongside other predictors, including sentiment, temporal signals, and textual features, when analysing patterns of viral engagement on social media platforms.

2.6 Tools Used in Prior Research

Across the literature, researchers commonly use established Python-based libraries such as Scikit-learn, Pandas, NumPy, and Statsmodels to collect, process, and model social media data. These tools support tasks including classification, regression, sentiment analysis, and time-series forecasting. A concise summary of the commonly used tools is provided in **Appendix A**.

2.7 Gaps in Existing Research and Study Contribution

Although research on social media virality is expanding, key gaps remain. Many studies rely on a single modelling approach, often treating sentiment, timing, or community features in isolation. Most empirical work also focuses on platforms such as TikTok, Instagram, or Twitter, leaving Reddit comparatively underexplored. Few studies integrate emotional, temporal, behavioural, and structural factors within one framework, despite evidence that virality emerges from their interaction. This project addresses these gaps by adopting a multi-method modelling framework that integrates text-based features, sentiment analysis, temporal variables, and community-level indicators within classification, regression, and time-series approaches to provide a more comprehensive and context-sensitive understanding of Reddit virality.

3 PROJECT DESIGN (1996 words)

This chapter outlines the design of the Reddit virality prediction project. It explains the project domain, system design requirements, overall architecture, technologies used, and the development and evaluation plan.

3.1 Domain and Users

This project focuses on the field of social media analytics, which analyses large volumes of user-generated data using computational techniques. Reddit provides a suitable environment for this study because its community-based structure produces diverse posting patterns, emotional expressions, and engagement behaviours that can be analysed using data mining and machine learning techniques.

The system developed in this project focuses on analysing engagement patterns and predicting viral posts using textual, temporal, sentiment, and community-level features derived from Reddit data. The design of the system is therefore influenced by the needs of users who wish to explore how viral behaviour emerges on the platform.

The primary user groups and their design needs are summarised in **Figure 3.1**.

User Group	Main Needs	How the System Supports Them	Example Use Case
Researchers in social media analytics	Understand how viral content develops and spreads within Reddit communities	The system provides organised datasets, predictive models, and network analysis tools that help researchers explore patterns behind viral engagement	A researcher analyses which subreddit communities are more likely to produce viral posts
Data analysts	Explore how different post features relate to engagement levels	Machine learning models and visualisations allow analysts to examine how factors such as sentiment, posting time, and subreddit activity influence engagement	An analyst studies whether posts published at certain times receive higher upvotes
Social media observers or content creators	Learn what factors may help posts gain more visibility	The system shows how content characteristics and posting behaviour are associated with higher engagement on Reddit	A content creator examines which wording or sentiment patterns appear more frequently in viral posts

Figure 3.1: User Groups and Their Design Requirements

3.2 User Needs and Design Requirements

A review of the domain and the objective of this project highlight several key requirements that guide the design of the system.

3.2.1 Clear organisation of data

Users require a clean and well-structured dataset that combines Reddit post data from the Kaggle dataset with derived features such as sentiment scores, temporal indicators, and subreddit-level statistics. Social media data is often noisy, incomplete, or inconsistent, making data preparation an essential step in the analysis process (Han, Pei, & Kamber, 2012). To address this, the system includes a structured data cleaning and preprocessing pipeline that ensures the dataset is suitable for modelling and analysis.

3.2.2 Interpretability

Users require more than accurate predictions, they also need to understand why a post is classified as viral or non-viral. Interpretability is important in applied machine learning because models should produce results that can be explained and understood by users (Molnar, 2022). The system therefore includes interpretable baseline models such as Logistic Regression alongside more complex models such as Random Forest and XGBoost, allowing users to compare predictive performance while still examining how key features influence predictions.

3.2.3 Support for multiple perspectives

Virality is influenced by interacting factors such as content, timing, and community structure. Research shows these elements work together to shape online visibility (Kumar et al., 2020). Because of this, the system is designed to support multiple feature groups and modelling approaches rather than relying on a single type of predictor.

3.2.4 Visual exploration

Visualisations help users interpret complex analytical results more effectively. Visual analytics techniques are commonly used to present machine learning outputs in a more understandable form (Keim et al., 2010). The system therefore includes charts, performance plots, and network visualisations that help users identify patterns in engagement and better interpret model results.

3.2.5 Reproducible workflow

A clear and structured workflow helps ensure that the analysis process remains transparent and repeatable. Machine learning frameworks such as Scikit-learn emphasise modular design and reproducibility when building analytical pipelines (Pedregosa et al., 2011). The system

therefore follows a reproducible workflow that integrates data preparation, feature engineering, modelling, and evaluation within a consistent analysis pipeline.

3.3 Overall System Structure

The project is organised as a structured analytical pipeline for studying Reddit engagement patterns. It begins with data collection and preparation, followed by feature engineering and predictive modelling. Additional steps, including temporal analysis, network-based exploration, and visualisation, help provide insights into how content, timing, sentiment, and community factors affect post visibility.

3.3.1 Data Gathering Approach

The primary data source used in this project is the publicly available Reddit Kaggle dataset (PrakharRathi25, n.d.), which contains post-level information collected from multiple subreddit communities. The dataset includes attributes such as post titles, text content, upvote counts, timestamps, and subreddit labels.

These attributes provide the core information required to analyse engagement patterns across different Reddit communities. Additional analytical variables are derived during preprocessing, including sentiment indicators and community-level measures that describe activity patterns within different subreddits.

3.3.2 Data Preparation and Feature Engineering

Before modelling is performed, the dataset undergoes preprocessing to ensure that the data is suitable for analysis. This stage prepares the dataset for modelling by organising the raw Reddit data into a structured format.

Following data preparation, feature engineering is conducted to derive meaningful predictors from the dataset. The features used in this project are organised into four main groups: textual features, temporal features, sentiment features, and community-level features.

Feature Category	Example Features (from dataset)	Description / Purpose
Post-level features	• Title length • Body length • Flair category • Sentiment score (VADER)	Captures characteristics of the post content and emotional tone that may influence user engagement.
Temporal features	• Hour of posting (from creation time) • Day of week • Month	Represents when the post was published. Posting time may affect visibility due to differences in user activity across time periods.
Community-level features	• Subreddit name • Posting frequency per subreddit (computed)	Captures the community context where the post appears. Different subreddits have different sizes, activity levels, and engagement behaviour.
Text-structural features	• Keyword presence extracted from title/body TF-IDF vectors from cleaned text	Represents the semantic content of the post. TF-IDF converts text into numerical features that allow machine learning models to analyse textual patterns.

Figure 3.2: Summary of feature categories used in the modelling pipeline

Textual features capture characteristics of post content, such as text length and word count. TF-IDF text vectorisation is also applied to convert cleaned text into numerical representations suitable for machine learning models. Temporal features describe when posts are published, while sentiment features represent the emotional tone of posts using VADER sentiment scores. Community-level features describe activity patterns within different subreddit communities.

During preprocessing, the target variables used for modelling are also defined. Posts are labelled as viral if their upvote count falls within the top 10% of the upvote distribution. In addition to this binary label used for classification, the raw upvote count is retained as a continuous variable for regression modelling.

Together, these features allow the system to capture multiple dimensions of engagement behaviour, including textual content, emotional tone, timing, and community activity.

3.3.3 Predictive Modelling Framework

The predictive modelling component of the system analyses engagement outcomes using both classification and regression approaches.
The classification task predicts whether a post is likely to become viral based on its characteristics. Several machine learning models are used for this purpose, including Logistic

Regression, Random Forest, and XGBoost classifiers. Logistic Regression serves as an interpretable baseline model, while Random Forest and XGBoost allow the system to capture more complex relationships between features.

In addition to classification, the system performs regression analysis to estimate the number of upvotes a post may receive. This task is implemented using Linear Regression, Random Forest, and XGBoost Regressor, which predict continuous engagement outcomes.

The dataset is divided into training and testing subsets to allow models to be trained and evaluated on separate data. During model development, cross-validation is applied within the training data to improve model robustness, and hyperparameter tuning is performed for ensemble models to identify effective model configurations.

3.3.4 Time-Series Component

The system design also includes a time-series analysis component to explore how posting activity and engagement patterns change over time on Reddit.

Temporal signals derived from the dataset, such as posting frequency across different hours, days, or months, are analysed to identify patterns in user activity. Aggregated engagement indicators are also examined to reveal temporal patterns related to post visibility.

Illustrative ARIMA-based demonstrations are included to show how time-series modelling techniques can be applied to Reddit activity data. These analyses provide insight into temporal trends in posting behaviour and engagement.

3.3.5 Network Analysis Component

To complement the predictive modelling framework, the system includes a network-inspired analysis that explores structural relationships within the dataset.

Topic clusters are constructed using TF-IDF representations and clustering techniques to group posts with similar textual characteristics. In addition, subreddit similarity graphs are generated based on linguistic similarities between subreddit content.

These graphs illustrate how communities are connected through shared themes and discussion topics. Structural indicators are then used to examine how communities relate to one another within the similarity network.

This analysis provides a simplified view of how discussions and communities are organised across Reddit and supports the project’s objective of examining how engagement patterns may emerge across related communities.

3.3.6 Prototype Design

The final component of the system design is the prototype implementation, which integrates the analytical components into a single workflow.

The prototype is implemented using a Jupyter Notebook environment, which allows code, visualisations, and explanatory text to be presented together within a single interface (Kluyver et al., 2016). This environment supports reproducible analysis and enables users to follow each stage of the modelling pipeline.

Visual outputs such as feature importance charts, time-series plots, and network diagrams are included to support interpretation of engagement patterns and modelling results.

3.4 Technologies and Tools

This project was implemented using the Python programming environment together with several libraries that support data processing, machine learning, statistical analysis, and visualisation. These tools enable the system to perform data preparation, predictive modelling, temporal analysis, and network analysis within a unified workflow.

Table 3.3 summarises the main technologies and tools used in the project.

Category	Tools	Description / Purpose
Programming Environment	• Python • Jupyter Notebook	Python provides the primary programming language used for data processing and machine learning implementation. Jupyter Notebook enables interactive development and allows code, analysis, and visual outputs to be presented together in a reproducible workflow.
Data Processing Libraries	• Pandas • NumPy	These libraries are used for data manipulation, dataset merging, numerical operations, and efficient handling of structured Reddit data.
Machine Learning Libraries	• Scikit-learn • XGBoost	Provide implementations of classification and regression algorithms, model training procedures, cross-validation methods, and hyperparameter tuning capabilities.
Analysis Tools	• VADER Sentiment Analysis • Statsmodels (ARIMA) • NetworkX	VADER is used to compute sentiment scores from Reddit text. Statsmodels supports time-series analysis through ARIMA modelling, while NetworkX enables construction and analysis of subreddit similarity networks.
Visualisation Tools	• Matplotlib Seaborn	These libraries generate visual outputs such as charts, distributions, and analytical plots that support interpretation of engagement patterns and modelling results.

Figure 3.3: Technologies and tools used in the system

3.5 Project Development Plan

The project followed a structured development plan covering stages such as dataset preparation, feature engineering, model development, evaluation, and report writing. A detailed project timeline, task breakdown, and Gantt chart illustrating the development schedule are provided in **Appendix B**.

3.6 Evaluation Strategy

The evaluation strategy outlines how the performance of the analytical models and methods developed in this project will be assessed. Since the project includes several analytical

components, different evaluation approaches are used for predictive modelling, time-series analysis, and network analysis.

For the predictive modelling component, separate evaluation metrics are applied for classification and regression tasks to reflect the different prediction objectives. These metrics allow the performance of the machine learning models to be systematically compared.

The time-series and network analysis components are evaluated through analysis of temporal patterns and structural relationships observed within the dataset. These evaluations focus on interpreting the insights generated from the models and visualisations.

3.6.1 Evaluation for Classification Models

The classification models aim to predict whether a Reddit post will become viral. Model performance is evaluated using standard classification metrics including accuracy, precision, recall, F1-score, and ROC-AUC.

These metrics provide a comprehensive assessment of classification performance, particularly for imbalanced datasets where viral posts represent a smaller proportion of the observations. The metrics allow the effectiveness of different classification algorithms to be compared and analysed.

3.6.2 Evaluation for Regression Models

The regression models are evaluated based on their ability to predict the number of upvotes received by a Reddit post. Model performance is assessed using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2).

These metrics measure prediction accuracy, the magnitude of prediction errors, and the explanatory power of the regression models.

3.6.3 Evaluation of Time-Series Analysis

The time-series component is evaluated by analysing temporal engagement patterns derived from the dataset. Visualisations are used to examine how Reddit engagement changes across different time periods.

In addition, ARIMA modelling is applied to analyse temporal trends in aggregated engagement data and to evaluate the ability of the model to capture underlying temporal dynamics.

3.6.4 Evaluation of Network Analysis

The network analysis component is evaluated through interpretation of the structural relationships observed in the subreddit similarity network.

Network visualisations are analysed to identify clusters of related communities, while centrality measures are examined to assess the relative structural importance of different subreddits within the network.

3.7 Data Constraints and Their Implications

The dataset used in this project contains post-level attributes including post titles, body text, upvote counts, comment counts, timestamps, and subreddit metadata. While these attributes provide useful information for analysing Reddit engagement patterns, several constraints related to data availability influenced the design of the analytical methods used in this system.

One major constraint concerns data access. Real-time Reddit data could not be collected through the Reddit API due to access restrictions and application approval requirements. In addition, the Pushshift service, which previously provided historical Reddit data access, is no longer publicly available. Consequently, the project relies on a static dataset obtained from Kaggle rather than continuously collected platform data.

The dataset structure also limits certain analytical approaches. The data does not include detailed engagement timelines such as time-stamped upvote or comment activity, which prevents modelling early engagement dynamics. Advanced sequence-based models such as Long Short-Term Memory (LSTM) networks are therefore unsuitable because they require continuous temporal sequences.

Similarly, the dataset does not contain user interaction structures such as reply networks, preventing full network diffusion analysis.

Despite these constraints, the dataset still supports analysis using textual, temporal, sentiment, and community features aligned with the multi-method approach in Template 1.2.

4 IMPLEMENTATION (2213 words)

The system was implemented as a multi-stage pipeline including data preparation, feature engineering, model training, and additional analytical components. Reddit posts were cleaned and processed before extracting textual, sentiment, temporal, and subreddit activity features. TF-IDF vectorisation was applied to convert text into numerical representations and combined with other features for modelling. Classification and regression models were trained to predict virality and engagement, alongside time-series and network analysis.

4.1 Dataset Preparation and Cleaning

The Reddit dataset consisted of multiple CSV files, each representing posts from a specific subreddit. These files were programmatically loaded and merged using Python to create a

single consolidated dataset. The resulting dataset contained 29,179 posts across 31 subreddit communities, including fields such as titles, body text, timestamps, upvote counts, and subreddit labels.

```
Columns:
Index(['Unnamed: 0', 'ID', 'is_Original', 'Flair', 'num_comments', 'Title', 'Subreddit', 'Body', 'URL', 'Upvotes', 'Comments', 'creation_date', 'is_orig
inal'], dtype='object')
Dataset shape (rows, columns): (29179, 13)
```

	Unnamed: 0	ID	is_Original	Flair	num_comments	Title	Subreddit	Body	URL	Upvotes	Comments	creation_date	is_original
0	0.0	fcpbui	False	NaN	13	Can't have sex t...	AdviceForTeens	I'm in a relatio...	https://www.redd...	24	Have you tried t...	2020-03-03 12:39:21	NaN
1	1.0	fbtk1w	False	NaN	3	How to break up ...	AdviceForTeens	I'd love for us ...	https://www.redd...	18	there's really t...	2020-03-01 20:44:48	NaN
2	2.0	1681jg	False	NaN	4	[MODPOST] [META]...	AdviceForTeens	Hey everyone! I'...	https://www.redd...	21	Me: http://i.img...	2013-01-09 10:23:30	NaN
3	3.0	fcmxds	False	NaN	0	The most importa...	AdviceForTeens	Here are some ti...	https://www.redd...	18	NaN	2020-03-03 09:33:01	NaN
4	4.0	f9gz1a	False	NaN	15	My dad found my ...	AdviceForTeens	So.. I got a vir...	https://www.redd...	18	How old are you?...	2020-02-26 04:51:52	NaN

Figure 4.1: Sample Preview of the Reddit Dataset

```
Dataset structure:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29179 entries, 0 to 29178
Data columns (total 13 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Unnamed: 0     26311 non-null  float64
 1   ID             29179 non-null  object
 2   is_Original    26311 non-null  object
 3   Flair          12636 non-null  object
 4   num_comments   29179 non-null  int64
 5   Title          29179 non-null  object
 6   Subreddit      29179 non-null  object
 7   Body           16405 non-null  object
 8   URL            29179 non-null  object
 9   Upvotes        29179 non-null  int64
 10  Comments       27163 non-null  object
 11  creation_date  29179 non-null  object
 12  is_original    2868 non-null   object
dtypes: float64(1), int64(2), object(10)
memory usage: 2.9+ MB
```

Figure 4.2: Structure and Summary of the Reddit Dataset

To prepare the data for modelling, irrelevant variables were removed and data types were standardised. The retained variables included post ID, title, body text, upvotes, creation date, subreddit name, and comment count. Upvote and comment counts were converted to numeric format, while timestamps were converted to datetime. Missing text fields were replaced with empty strings, and duplicate posts were removed based on post identifiers (**Appendix C**). After cleaning, the dataset contained 28,336 posts across 30 subreddits.

```
Final columns used for modelling:
Index(['ID', 'Title', 'Body', 'Upvotes', 'creation_date', 'Subreddit',
       'num_comments'],
      dtype='object')
Dataset shape (rows, columns): (28336, 7)
```

	ID	Title	Body	Upvotes	creation_date	Subreddit	num_comments
0	fcpbui	Can't have sex the same	I'm in a relationship of 2 years, naturally be...	24	2020-03-03 12:39:21	AdviceForTeens	13
1	fbtk1w	How to break up with a friend in the most peac...	I'd love for us to slide off of each other but...	18	2020-03-01 20:44:48	AdviceForTeens	3
2	1681jg	[MODPOST] [META] Lets bring this subreddit bac...	Hey everyone! I'm one of your friendly neighbo...	21	2013-01-09 10:23:30	AdviceForTeens	4
3	fcmxds	The most important thing when texting a girl i...	Here are some tips on how to text a girl and s...	18	2020-03-03 09:33:01	AdviceForTeens	0
4	f9gz1a	My dad found my porn, help	So.. I got a virus onto my laptop so my dad de...	18	2020-02-26 04:51:52	AdviceForTeens	15

Figure 4.3: Preview of the Cleaned Reddit Dataset

```
Dataset structure:
<class 'pandas.core.frame.DataFrame'>
Index: 28336 entries, 0 to 29178
Data columns (total 7 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   ID             28336 non-null  object
 1   Title          28336 non-null  object
 2   Body           28336 non-null  object
 3   Upvotes        28336 non-null  int64
 4   creation_date  28336 non-null  datetime64[ns]
 5   Subreddit      28336 non-null  object
 6   num_comments   28336 non-null  int64
dtypes: datetime64[ns](1), int64(2), object(4)
memory usage: 1.7+ MB
```

Figure 4.4: Structure of the Cleaned Dataset After Preprocessing

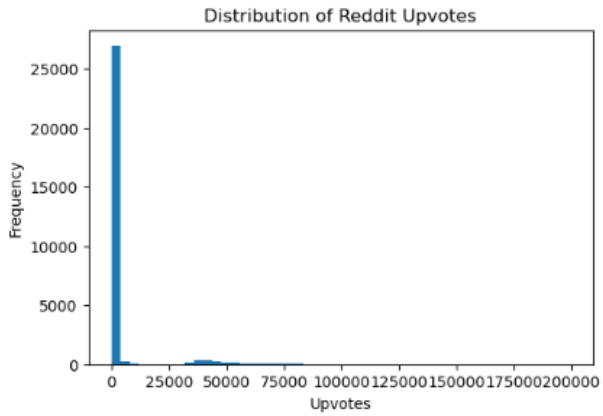
To identify viral posts, the distribution of upvotes was examined and found to be highly right-skewed, meaning most posts receive low engagement while a small number receive extremely high upvotes.

```python
# Plot histogram to visualise distribution of upvotes
plt.figure(figsize=(6,4))
plt.hist(df["Upvotes"], bins=50)
plt.title("Distribution of Reddit Upvotes")
plt.xlabel("Upvotes")
plt.ylabel("Frequency")
plt.show()

# Calculate descriptive statistics for upvotes
stats = df["Upvotes"].describe()

# Measure skewness to check if distribution is highly right-skewed
skewness = df["Upvotes"].skew()

print("Upvote statistics:")
print(stats)
print("Skewness:", skewness)
```




```
Upvote statistics:
count     29179.000000
mean       3862.969156
std       13000.969348
min           0.000000
25%          15.000000
50%         296.000000
75%         897.000000
max      199296.000000
Name: Upvotes, dtype: float64
Skewness: 4.2277244668448155
```

Figure 4.5: Distribution of Reddit Upvotes and Descriptive Statistics

Based on this distribution, the 90th percentile of upvotes (2,391 upvotes) was selected as the viral threshold. Posts with upvote counts above this value were labelled as viral (is_viral = 1), while the remaining posts were labelled as non-viral (is_viral = 0). The resulting class distribution is shown in **Figure 4.6.**

```python
# Define viral threshold using the 90th percentile of upvotes
viral_threshold = df["Upvotes"].quantile(0.90)

# Display the calculated threshold value
print("Viral threshold (90th percentile):", viral_threshold)

# Create binary viral label based on the threshold
# 1 = viral (top 10%), 0 = non-viral
df["is_viral"] = (df["Upvotes"] >= viral_threshold).astype(int)

# Check class distribution (counts)
print("Viral label distribution:")
print(df["is_viral"].value_counts())

# Check class distribution (proportions)
print(df["is_viral"].value_counts(normalize=True))
```

```
Viral threshold (90th percentile): 2391.0
Viral label distribution:
is_viral
0    25502
1     2834
Name: count, dtype: int64
is_viral
0    0.899986
1    0.100014
Name: proportion, dtype: float64
```

Figure 4.6: Viral Threshold Determination and Class Distribution

The processed dataset was then used for feature engineering and machine learning modelling.

4.2 Text Processing and Feature Engineering

4.2.1 Text Preprocessing

Before feature extraction, the textual content of Reddit posts was preprocessed to improve data consistency. Each post contained two textual fields: the title and body text, which were combined into a single field representing the full content of the post. Text normalisation was then applied by converting all characters to lowercase so that words with different capitalisation were treated consistently during feature extraction. Additional cleaning steps removed noise from the text, including URLs, HTML tags, special symbols, standalone numbers, and extra whitespace. The implementation details of the text preprocessing procedures are provided in **Appendix D**.

	Before Cleaning	After Cleaning
0	Can't have sex the same I'm in a relationship ...	can't have sex the same i'm in a relationship ...
2	[MODPOST] [META] Lets bring this subreddit bac...	[modpost] [meta] lets bring this subreddit bac...
3	The most important thing when texting a girl i...	the most important thing when texting a girl i...
5	How to tell my parents I want to get medical a...	how to tell my parents i want to get medical a...
6	Golden advice that can help you 1. If you wast...	golden advice that can help you . if you waste...
7	If you need advice I've got some for you If yo...	if you need advice i've got some for you if yo...
8	Everything's changing I'm 17 and a senior, so ...	everything's changing i'm and a senior, so you...
9	I'm 14 and I lied about having a gf and a girl...	i'm and i lied about having a gf and a girl wi...
10	Advice on crushes? It's pretty basic to have t...	advice on crushes? it's pretty basic to have t...
11	I'm 13 and I still play with toys, and everyon...	i'm and i still play with toys, and everyone (...

Figure 4.7: Comparison of Text Data Before and After Cleaning

4.2.2 Feature Engineering

Following preprocessing, several features were engineered from the dataset. Text length features were created, including total text length, word count, title length, and body length.

```
# total number of characters in cleaned text
df["text_length"] = df["text_clean"].str.len()

# total number of words in cleaned text
df["word_count"] = df["text_clean"].str.split().apply(len)

# measure title and body length separately
df["title_length"] = df["Title"].str.len()
df["body_length"] = df["Body"].str.len()

# Quick check of created features
print("Text feature columns created:",
      ["text_length", "word_count", "title_length", "body_length"])

df[["text_length", "word_count", "title_length", "body_length"]].head()
```

```
Text feature columns created: ['text_length', 'word_count', 'title_length', 'body_length']
```

	text_length	word_count	title_length	body_length
0	619	116	23	601
1	841	169	64	789
2	1293	237	101	1289
3	302	65	246	84
4	425	87	26	402

```
df[["text_length","word_count","Upvotes","Subreddit"]].head()
```

	text_length	word_count	Upvotes	Subreddit
0	619	116	24	AdviceForTeens
1	841	169	18	AdviceForTeens
2	1293	237	21	AdviceForTeens
3	302	65	18	AdviceForTeens
4	425	87	18	AdviceForTeens

Figure 4.8: Generated Text-Based Features

Sentiment features were generated using the VADER sentiment analyser, producing negative, neutral, positive, and compound sentiment scores for each post.

```python
# Initialise VADER
sia = SentimentIntensityAnalyzer()

# Function to extract sentiment scores
def get_sentiment_scores(text):
    scores = sia.polarity_scores(text)
    return pd.Series([
        scores["neg"],
        scores["neu"],
        scores["pos"],
        scores["compound"]
    ])

# Apply to cleaned text
df[["sent_neg", "sent_neu", "sent_pos", "sent_compound"]] = df["text_clean"].apply(get_sentiment_scores)

# Quick check
print("Sentiment features created.")
df[["text_clean", "sent_neg", "sent_neu", "sent_pos", "sent_compound"]].head()
```

```
Sentiment features created.
```

	text_clean	sent_neg	sent_neu	sent_pos	sent_compound
0	can't have sex the same i'm in a relationship ...	0.209	0.769	0.022	-0.9808
1	how to break up with a friend in the most peac...	0.050	0.855	0.095	0.6794
2	[modpost] [meta] lets bring this subreddit bac...	0.085	0.761	0.155	0.9322
3	the most important thing when texting a girl i...	0.104	0.816	0.080	-0.3832
4	my dad found my porn, help so.. i got a virus ...	0.038	0.815	0.146	0.8890

Figure 4.9: Sentiment Feature Extraction Using VADER

Temporal features were derived from the post timestamp, including posting hour, day of the week, month, and a weekend indicator (**Appendix E**). In addition, a community-level feature (subreddit_post_count) was computed after the train–test split (Section 4.3.2) to prevent data leakage.

These engineered features were generated as numerical variables for use in the subsequent machine learning modelling stages.

4.3 Text Vectorisation and Data Preparation for Modelling

4.3.1 Train–Test Split

Before model training, the dataset was divided into training (80%, 22,668 posts) and testing (20%, 5,668 posts) sets using a stratified train–test split. Stratification preserved the proportion of viral and non-viral posts across both datasets, ensuring a representative split for

training and evaluation on unseen data (**Figure 4.10**).

```python
from sklearn.model_selection import train_test_split

# Split the dataframe
df_train, df_test = train_test_split(
    df,
    test_size=0.20,
    random_state=42,
    stratify=df["is_viral"]
)

print("Train df shape:", df_train.shape)
print("Test df shape:", df_test.shape)

print("\nTrain class distribution:")
print(df_train["is_viral"].value_counts(normalize=True))

print("\nTest class distribution:")
print(df_test["is_viral"].value_counts(normalize=True))
```

```
Train df shape: (22668, 24)
Test df shape: (5668, 24)

Train class distribution:
is_viral
0    0.899991
1    0.100009
Name: proportion, dtype: float64

Test class distribution:
is_viral
0    0.899965
1    0.100035
Name: proportion, dtype: float64
```

Figure 4.10: Train-Test Split and Class Distribution of Viral Labels

4.3.2 Community-level Feature Construction and Text Vectorisation using TF-IDF

After splitting the dataset, a community-level feature (subreddit_post_count) was created to represent subreddit activity and applied to both the training and testing sets.

```python
# Count subreddit activity using TRAIN only
train_sub_counts = df_train["Subreddit"].value_counts()

# Add feature to train and test by mapping from TRAIN counts
df_train["subreddit_post_count"] = df_train["Subreddit"].map(train_sub_counts)
df_test["subreddit_post_count"] = df_test["Subreddit"].map(train_sub_counts)

# If any subreddit appears only in test, fill with 0
df_test["subreddit_post_count"] = df_test["subreddit_post_count"].fillna(0)

print("subreddit_post_count added safely (train-only counts).")
print("Train range:", df_train["subreddit_post_count"].min(), "to", df_train["subreddit_post_count"].max())
print("Test range:", df_test["subreddit_post_count"].min(), "to", df_test["subreddit_post_count"].max())
```

```
subreddit_post_count added safely (train-only counts).
Train range: 38 to 1989
Test range: 38 to 1989
```

```python
df_train[["Subreddit", "subreddit_post_count"]].head()
df_test[["Subreddit", "subreddit_post_count"]].head()
```

	Subreddit	subreddit_post_count
11299	dating	809
9579	COVID19	823
12619	jobs	1989
20219	kidsrights	382
793	AdviceForTeens	658

Figure 4.11: Subreddit Activity Feature Generation Using Training Data

The cleaned Reddit text was converted into numerical features using TF-IDF vectorisation, configured with 5,000 features, stopword removal, and an n-gram range of (1,2).

```
tfidf = TfidfVectorizer(
    max_features=5000,       # keep top 5000 important words
    stop_words='english',    # remove common English stopwords
    ngram_range=(1, 2),       # use unigrams + bigrams
)

print("TF-IDF vectoriser initialised.")
```

```
TF-IDF vectoriser initialised.
```

Figure 4.12: TF-IDF Vectoriser Configuration

The vectoriser was fitted on the training data and applied to the test data to avoid leakage.

```
# TF-IDF: fit on TRAIN only, transform TRAIN and TEST
train_text = df_train["text_clean"].astype(str)
test_text  = df_test["text_clean"].astype(str)

X_train_tfidf = tfidf.fit_transform(train_text)
X_test_tfidf  = tfidf.transform(test_text)

print("TF-IDF train shape:", X_train_tfidf.shape)
print("TF-IDF test shape:", X_test_tfidf.shape)
```

```
TF-IDF train shape: (22668, 5000)
TF-IDF test shape: (5668, 5000)
```

Figure 4.13: TF-IDF Transformation Results

4.3.4 Feature Scaling and Final Feature Matrix

Numerical features derived from earlier feature engineering, including text length, sentiment, temporal, and community-level variables, were standardised using StandardScaler and combined with TF-IDF features to form the final feature matrix.

```
from scipy.sparse import hstack
from sklearn.preprocessing import StandardScaler

numeric_features = [
    "text_length", "word_count", "title_length", "body_length",
    "sent_neg", "sent_neu", "sent_pos", "sent_compound",
    "num_comments",
    "post_hour", "post_dayofweek", "post_month", "is_weekend",
    "subreddit_post_count"
]

# Numeric features for train/test
X_train_num = df_train[numeric_features].values
X_test_num  = df_test[numeric_features].values

# Scale numeric features (fit on TRAIN only)
scaler = StandardScaler()
X_train_num_scaled = scaler.fit_transform(X_train_num)
X_test_num_scaled  = scaler.transform(X_test_num)

# Combine TF-IDF + scaled numeric
X_train = hstack([X_train_tfidf, X_train_num_scaled])
X_test  = hstack([X_test_tfidf, X_test_num_scaled])

# Targets
y_train = df_train["is_viral"].astype(int).values
y_test  = df_test["is_viral"].astype(int).values

print("Final X_train shape:", X_train.shape)
print("Final X_test shape:", X_test.shape)
print("y_train:", y_train.shape, "y_test:", y_test.shape)
```

```
Final X_train shape: (22668, 5014)
Final X_test shape: (5668, 5014)
y_train: (22668,) y_test: (5668,)
```

Figure 4.13: Feature Scaling and Combination of TF-IDF and Numerical

4.4 Machine Learning Model Implementation

4.4.1 Classification Models (RQ1)

Three classifiers were implemented to predict whether Reddit posts are viral or non-viral. All models were trained using the feature matrix from Section 4.3. Model performance was evaluated using accuracy, precision, recall, F1-score, and ROC-AUC, given the imbalanced distribution of classes.

4.4.2.1 Logistic Regression

A baseline Logistic Regression classifier was implemented using the LogisticRegression class from the scikit-learn library. Logistic Regression was used as an initial baseline model for comparison with more complex classifiers.

The model was fitted to the training dataset using the implementation shown below.

```
# Initialize baseline model
logreg = LogisticRegression(
    max_iter=1000,
    class_weight="balanced",
    random_state=42
)

# Train on training set
logreg.fit(X_train, y_train)
```

Figure 4.14: Logistic Regression Model Training

Predictions were then generated on the test dataset using the trained model to evaluate classification performance.

```
# Predict on test set
y_pred_lr_clf = logreg.predict(X_test)
y_prob_lr_clf = logreg.predict_proba(X_test)[:, 1]
```

Figure 4.15: Logistic Regression Prediction on Test Data

The classification report, confusion matrix, and ROC curve produced from the model predictions are shown in **Figure 4.16**.

```
Logistic Regression Results

              precision    recall  f1-score   support

           0       0.98      0.96      0.97      5101
           1       0.69      0.85      0.76       567

    accuracy                           0.95      5668
   macro avg       0.83      0.90      0.86      5668
weighted avg       0.95      0.95      0.95      5668

Confusion Matrix:
[[4882  219]
 [  87  480]]

ROC-AUC: 0.9750
```

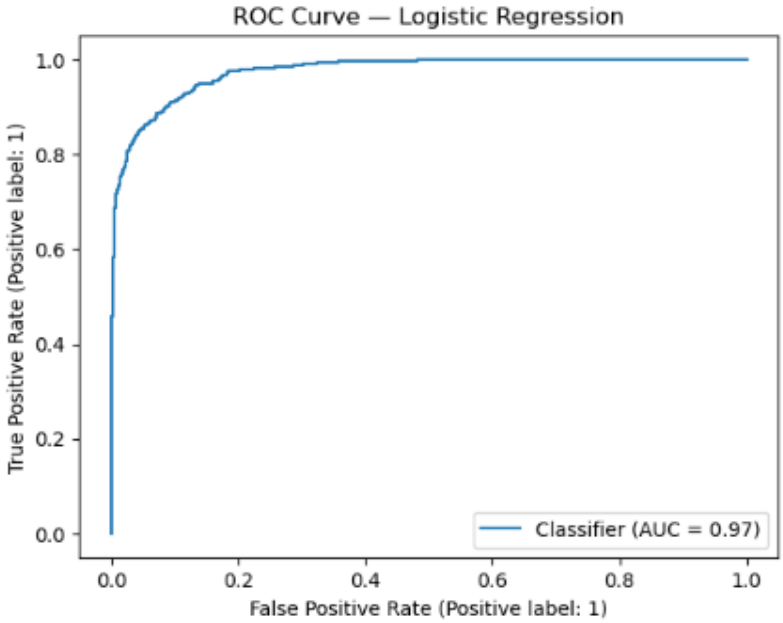


Figure 4.16: Logistic Regression Model Results

To further assess model stability, Stratified 5-Fold Cross-Validation was applied to the training dataset using the cross_validate() function in scikit-learn. This method ensures that the proportion of viral and non-viral posts is preserved across validation folds.

```python
# Reuse the same baseline model
logreg_cv = LogisticRegression(
    max_iter=1000,
    class_weight="balanced",
    random_state=42
)

# Stratified 5-fold CV
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Evaluate multiple metrics
cv_results = cross_validate(
    logreg_cv,
    X_train,
    y_train,
    cv=cv,
    scoring=["accuracy", "precision", "recall", "f1", "roc_auc"],
    n_jobs=-1
)
```

Figure 4.17: Stratified 5-Fold Cross-Validation Setup for Logistic Regression

The cross-validation results show consistent performance across the validation folds, indicating stable model behaviour.

```
Cross-Validation Results (Logistic Regression)

accuracy: 0.9445 ± 0.0027
precision: 0.6841 ± 0.0146
recall: 0.8289 ± 0.0147
f1: 0.7494 ± 0.0108
roc_auc: 0.9648 ± 0.0045
```

Figure 4.18: Logistic Regression Cross-Validation Results

Logistic Regression therefore serves as a baseline classifier for distinguishing viral and non-viral Reddit posts.

4.4.2.2 Random Forest Classifier

A Random Forest classifier was implemented using the RandomForestClassifier class from scikit-learn to model more complex relationships between features and post virality.

The same training and prediction pipeline described in **Section 4.4.2.1** was applied, with classification results shown in **Figure 4.18**

```
Random Forest Classifier Baseline Results

              precision    recall  f1-score   support

           0       0.96      1.00      0.98      5101
           1       0.99      0.66      0.80       567

    accuracy                           0.97      5668
   macro avg       0.98      0.83      0.89      5668
weighted avg       0.97      0.97      0.96      5668

Confusion Matrix:
[[5099    2]
 [ 190  377]]

ROC-AUC: 0.9647
```

Figure 4.19: Baseline Random Forest Classifer Model Results

To improve model performance, hyperparameter tuning was performed using GridSearchCV with Stratified 5-Fold Cross-Validation, exploring parameters including the number of trees, maximum tree depth, and minimum samples required for node splitting.

```python
# Base model
rf = RandomForestClassifier(
    random_state=42,
    n_jobs=-1
)

# Parameter grid
param_grid_rf = {
    "n_estimators": [100, 200],
    "max_depth": [None, 20],
    "min_samples_split": [2, 5]
}

# Grid search with stratified CV
grid_rf = GridSearchCV(
    rf,
    param_grid_rf,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring="f1",
    n_jobs=-1,
    verbose=1
)

# Fit on TRAINING DATA ONLY
grid_rf.fit(X_train, y_train)
```

Figure 4.19: GridSearchCV Setup for Random Forest Classifer

The optimal parameter configuration identified by GridSearchCV is shown below.

```
Fitting 5 folds for each of 8 candidates, totalling 40 fits
Best RF parameters: {'max_depth': None, 'min_samples_split': 2, 'n_estimators': 100}
Best RF CV score: 0.7960320985705949
```

Figure 4.20: GridSearchCV Setup for Random Forest Classifer

The tuned model was then refitted using the optimal parameters and evaluated on the test dataset.

```
Tuned Random Forest Classifier Test Results

              precision    recall  f1-score   support

           0       0.96      1.00      0.98      5101
           1       0.99      0.66      0.80       567

    accuracy                           0.97      5668
   macro avg       0.98      0.83      0.89      5668
weighted avg       0.97      0.97      0.96      5668

Confusion Matrix:
[[5099    2]
 [ 190  377]]

ROC-AUC: 0.9647
```

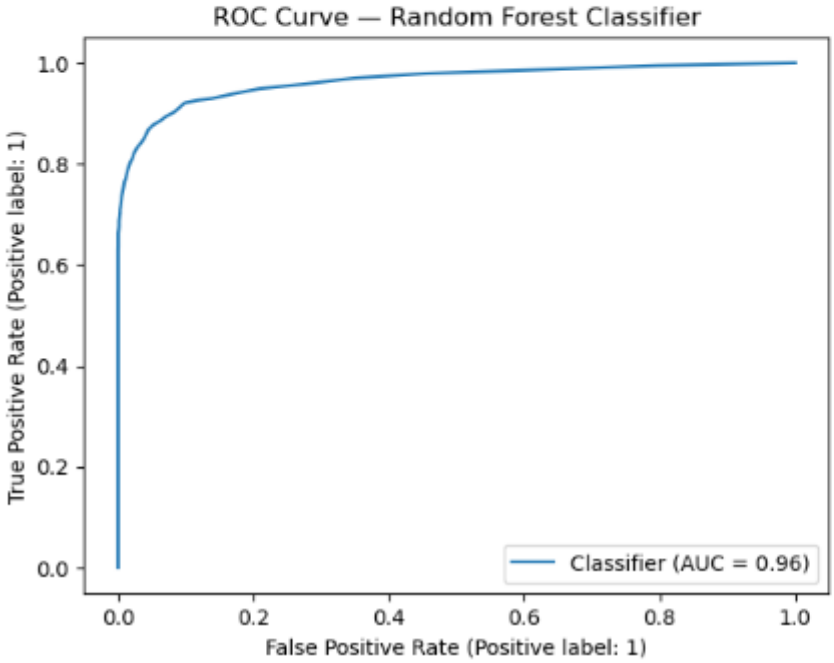


Figure 4.21: Tuned Random Forest Classifer Model Results

The results show that Random Forest improves predictive performance compared to the baseline Logistic Regression model.

4.4.2.3 XGBoost Classifier

An XGBoost classifier was implemented using the XGBClassifier class from the XGBoost library to evaluate the performance of gradient-boosted decision trees.

To address class imbalance, the scale_pos_weight parameter was calculated from the training data based on class distribution and incorporated into the model.

The XGBoost model was trained on the training dataset and used to predict on the test dataset, with results shown in **Figure 4.18**.

```
scale_pos_weight: 8.999
XGBoost CLassifier Baseline Results

              precision    recall  f1-score   support

           0       0.99      0.95      0.97      5101
           1       0.66      0.93      0.77       567

    accuracy                           0.95      5668
   macro avg       0.83      0.94      0.87      5668
weighted avg       0.96      0.95      0.95      5668

Confusion Matrix:
[[4835  266]
 [  41  526]]

ROC-AUC: 0.9872
```

Figure 4.22: Baseline XGBoost Classifer Results

Hyperparameter tuning was performed using GridSearchCV with Stratified 5-Fold Cross-Validation.

```python
# Base model
xgb = XGBClassifier(
    random_state=42,
    scale_pos_weight=scale_pos_weight,
    eval_metric="logloss",
    n_jobs=-1
)

# Parameter grid
param_grid_xgb = {
    "n_estimators": [200, 400],
    "max_depth": [3, 5],
    "learning_rate": [0.1],
    "subsample": [0.8],
    "colsample_bytree": [0.8]
}

# Grid search with stratified CV
grid_xgb = GridSearchCV(
    xgb,
    param_grid_xgb,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring="f1",
    n_jobs=-1,
    verbose=1
)

# Fit on TRAIN only
grid_xgb.fit(X_train, y_train)
```

Figure 4.23: GridSearchCV Setup for XGBoost Classifer

The tuned model was subsequently evaluated on the test dataset using the optimal parameter configuration.

```
Fitting 5 folds for each of 4 candidates, totalling 20 fits

Best XGB parameters: {'colsample_bytree': 0.8, 'learning_rate': 0.1, 'max_depth': 5, 'n_estimators': 400, 'subsample': 0.8}
Best XGB CV score: 0.8396084605333101
```

Figure 4.24: Optimal Hyperparameters (XGBoost Classifer)

```
Tuned XGBoost Classifier Test Results

              precision    recall  f1-score   support

           0       0.99      0.97      0.98      5101
           1       0.79      0.88      0.84       567

    accuracy                           0.97      5668
   macro avg       0.89      0.93      0.91      5668
weighted avg       0.97      0.97      0.97      5668

Confusion Matrix:
[[4972  129]
 [  68  499]]

ROC-AUC: 0.9867
```

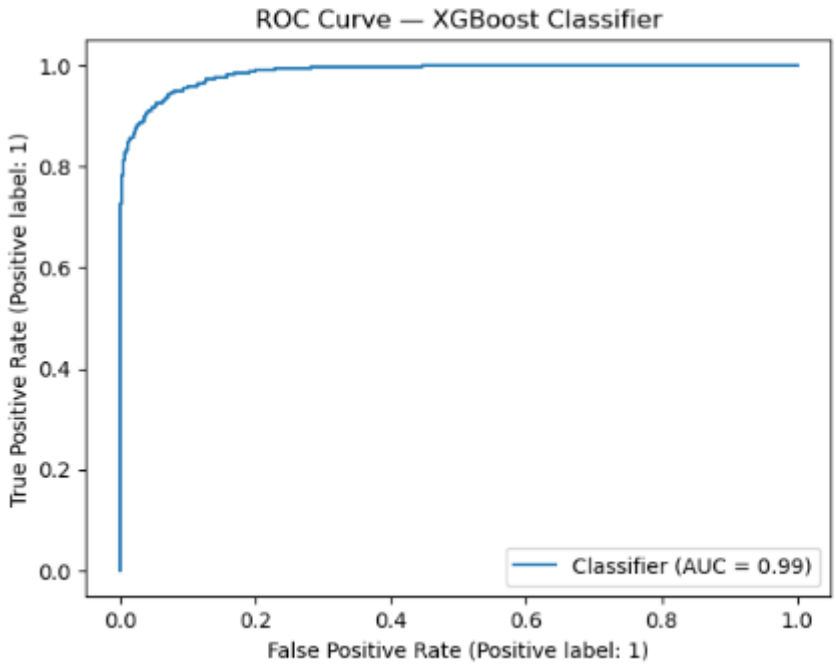


Figure 4.25: Tuned Random Forest Classifer Model Results

Among the evaluated classifiers, the tuned XGBoost model produced the strongest classification performance, demonstrating the effectiveness of boosted ensemble methods for predicting Reddit post virality.

4.4.2 Regression Models (RQ2)

Regression models were used to predict Reddit post upvotes as a continuous variable, unlike classification models that predict viral outcomes. Performance was evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score.

4.4.2.1 Regression Target Preparation

Before training, the Reddit upvotes target variable was log-transformed using NumPy's log1p function to address its right-skewed distribution and improve model learning.

```
# Log-transform upvotes to reduce skewness for regression
df["log_upvotes"] = np.log1p(df["Upvotes"])
```

Figure 4.26: Log Transformation of Upvotes Target Variable

The transformed variable (log_upvotes) was then split using the same train–test setup, resulting in 22,668 training and 5,668 testing observations.

```
y_reg_train = df_train["log_upvotes"].values
y_reg_test  = df_test["log_upvotes"].values
```

Figure 4.27: Train–Test Split of Transformed Regression Target

4.4.2.2 Linear Regression

A baseline Linear Regression model was implemented using the LinearRegression class from the scikit-learn library. Linear Regression was used as a baseline regression model for predicting the log-transformed upvote counts.

The model was trained and used to predict on the test dataset, with performance evaluated using MAE, RMSE, and R^2, as shown in **Figure 4.28**.

```
Linear Regression Results

MAE:  1.5388
RMSE: 1.9968
R²:   0.4808
```

Figure 4.28: Linear Regression Results

To further assess model stability, 5-Fold Cross-Validation was applied using the cross_validate() function in scikit-learn.

```python
# Define CV strategy
cv = KFold(n_splits=5, shuffle=True, random_state=42)

# Linear regression for CV
linreg_cv = LinearRegression()

# Run cross-validation
cv_results_reg = cross_validate(
    linreg_cv,
    X_train,
    y_reg_train,
    cv=cv,
    scoring=["neg_mean_absolute_error",
             "neg_root_mean_squared_error",
             "r2"],
    n_jobs=-1
)
```

Figure 4.29: 5-Fold Cross-Validation Setup for Linear Regression

The cross-validation results show consistent regression performance across validation folds, indicating stable model behaviour.

```
Cross-Validation Results (Linear Regression)

MAE:  1.6207 ± 0.0143
RMSE: 2.1190 ± 0.0332
R²:   0.4086 ± 0.0302
```

Figure 4.29: Linear Regression Cross-Validation Results

Linear Regression therefore serves as a baseline model for predicting log-transformed Reddit upvote counts.

4.4.2.3 Random Forest Regressor

A Random Forest Regressor was implemented using the RandomForestRegressor class from scikit-learn to model more complex relationships between the input features and Reddit engagement levels.

The baseline model was trained and used to predict on the test dataset, with performance evaluated using MAE, RMSE, and R^2, as shown in **Figure 4.30**.

```
Baseline Random Forest Regressor Results

MAE:  0.4280
RMSE: 0.6018
R²:   0.9528
```

Figure 4.30: Baseline Random Forest Regressor Results

To improve predictive performance, hyperparameter tuning was conducted using GridSearchCV with 5-Fold Cross-Validation, exploring parameters such as the number of trees, tree depth, and minimum samples required for node splitting.

```
# Base model
rf_reg = RandomForestRegressor(
    random_state=42,
    n_jobs=-1
)

# Parameter grid
param_grid_rf_reg = {
"n_estimators": [50],
"max_depth": [15],
"min_samples_split": [5],
"min_samples_leaf": [2]
}

# Grid search with k-fold CV
grid_rf_reg = GridSearchCV(
    estimator=rf_reg,
    param_grid=param_grid_rf_reg,
    cv=KFold(n_splits=5, shuffle=True, random_state=42),
    scoring="neg_root_mean_squared_error",
    n_jobs=-1,
    verbose=2
)
# Fit on TRAINING DATA ONLY
grid_rf_reg.fit(X_train, y_reg_train)
```

Figure 4.31: GridSearchCV Setup for Random Forest Regressor

The optimal parameter configuration identified by GridSearchCV is shown below.

```
Fitting 5 folds for each of 1 candidates, totalling 5 fits

Best RF Regressor parameters: {'max_depth': 15, 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 50}
Best RF Regressor CV score (neg RMSE): -0.6205596390861561
```

Figure 4.32: Optimal Hyperparameters (Random Forest Regressor)

The tuned model was then refitted using the optimal parameters and evaluated on the test dataset.

```
Tuned Random Forest Regressor Test Results

MAE:  0.4288
RMSE: 0.5988
R²:   0.9533
```

Figure 4.33: Tuned Random Forest Regressor Results

The results indicate that Random Forest improves predictive accuracy compared to the baseline Linear Regression model.

4.4.2.4 XGBoost Regressor

An XGBoost Regressor was implemented to model non-linear relationships in engagement prediction. The model was trained and used to predict on the test dataset, with performance evaluated using MAE, RMSE, and R^2 (**Figure 4.32**). Hyperparameter tuning was applied, and the tuned model achieved strong performance comparable to Random Forest. Full implementation details are provided in **Appendix E**.

4.4.3 Time-Series Analysis of Reddit Engagement (RQ3)

To address Research Question 3, a time-series analysis was conducted to examine how Reddit engagement evolved over time and whether short-term forecasting could capture these dynamics. As discussed in the project design, this component focuses on aggregated engagement patterns rather than a full real-time forecasting system.

First, the creation_date column was converted to datetime format, sorted chronologically, and set as the dataframe index to prepare the data for time-series analysis. This ensured that the observations were ordered correctly for temporal modelling.

```
Missing timestamps: 0
Time column prepared
```

creation_date	Upvotes	Subreddit	num_comments
2008-03-17 06:00:25	710	business	268
2008-03-17 19:07:24	972	business	109
2008-03-18 17:36:39	576	business	422
2008-07-02 15:40:33	563	business	234
2008-07-29 12:18:15	746	business	125
2008-09-09 22:11:19	629	business	107
2008-09-16 23:28:12	1090	business	748
2008-09-19 10:49:34	627	business	227
2008-09-22 13:59:30	973	business	243
2008-09-23 23:37:52	1811	business	228

Figure 4.34: Sample of Reddit dataset after datetime conversion and chronological sorting

Next, weekly total upvotes were constructed by resampling the dataset at weekly intervals and summing the upvote counts within each week. Missing weeks were filled with zero values to maintain a continuous time series. The resulting weekly series is shown in Figure **4.35**, which illustrates a strong upward trend in Reddit engagement over time, particularly after 2017.

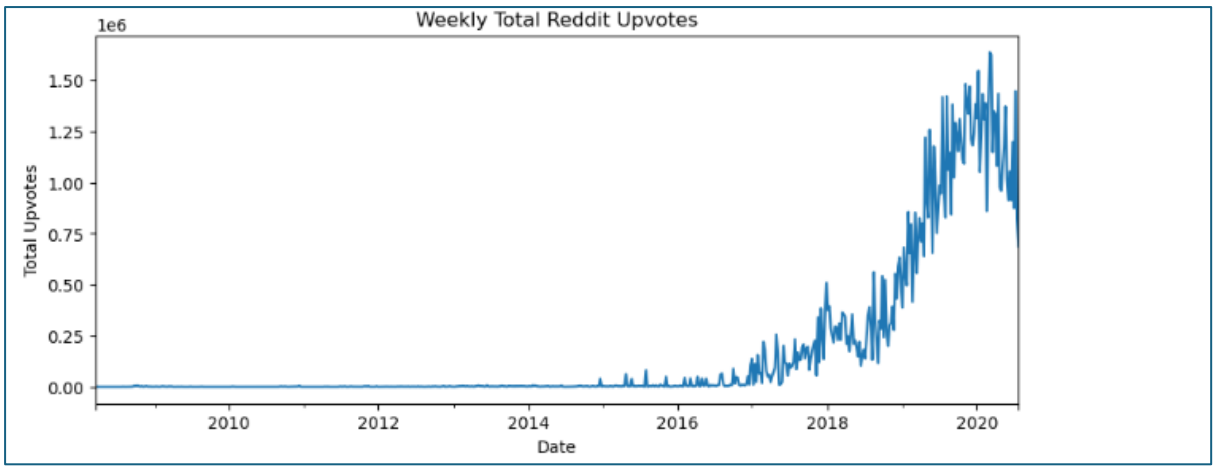


Figure 4.35: Weekly total Reddit upvotes overtime

To assess stationarity, the Augmented Dickey–Fuller (ADF) test was applied to the weekly time series.

```
# run ADF test on WEEKLY series
adf_result = adfuller(ts_weekly)

print("ADF Statistic:", adf_result[0])
print("p-value:", adf_result[1])

# interpretation helper
if adf_result[1] < 0.05:
    print("Series is stationary (good for ARIMA)")
else:
    print("Series is NOT stationary (need differencing)")
```

```
ADF Statistic: -1.6665544519624544
p-value: 0.44842174560882087
Series is NOT stationary (need differencing)
```

Figure 4.36: ADF test results for stationarity check

The initial test indicated that the series was non-stationary, so first-order differencing was performed.

```python
# apply first difference
ts_weekly_diff = ts_weekly.diff().dropna()

print("Differenced series length:", len(ts_weekly_diff))

# plot differenced series
import matplotlib.pyplot as plt

plt.figure(figsize=(10,4))
ts_weekly_diff.plot(title="Weekly Upvotes (First Difference)")
plt.xlabel("Date")
plt.ylabel("Differenced Upvotes")
plt.show()
```

```
Differenced series length: 644
```

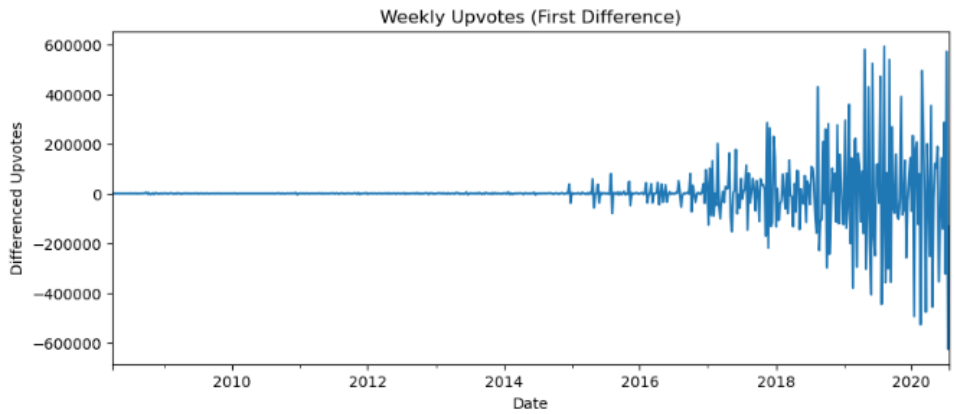


Figure 4.37: Differenced weekly Reddit upvotes

A second ADF test confirmed that the differenced series was stationary and therefore suitable for ARIMA modelling.

```python
adf_result_diff = adfuller(ts_weekly_diff)

print("ADF Statistic:", adf_result_diff[0])
print("p-value:", adf_result_diff[1])

if adf_result_diff[1] < 0.05:
    print("Differenced series is stationary")
else:
    print("Still not stationary — may need second differencing")
```

```
ADF Statistic: -3.045784180989611
p-value: 0.0308343524892797
Differenced series is stationary
```

Figure 4.38: Second ADF test confirming stationarity

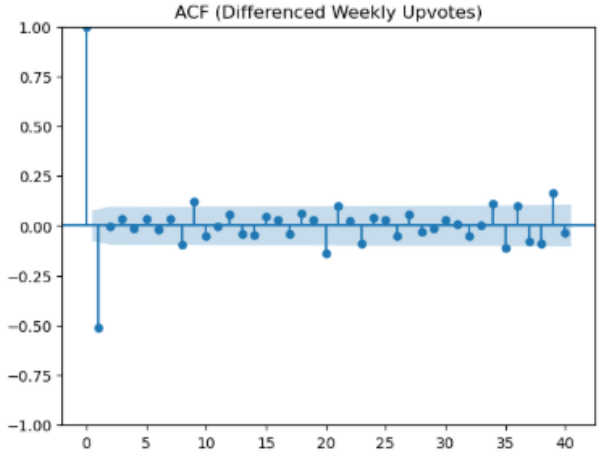
Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) plots were then examined to guide ARIMA order selection.

```python
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt

plt.figure(figsize=(10,4))
plot_acf(ts_weekly_diff, lags=40)
plt.title("ACF (Differenced Weekly Upvotes)")
plt.show()

plt.figure(figsize=(10,4))
plot_pacf(ts_weekly_diff, lags=40, method="ywm")
plt.title("PACF (Differenced Weekly Upvotes)")
plt.show()
```

```
<Figure size 1000x400 with 0 Axes>
```




```
<Figure size 1000x400 with 0 Axes>
```

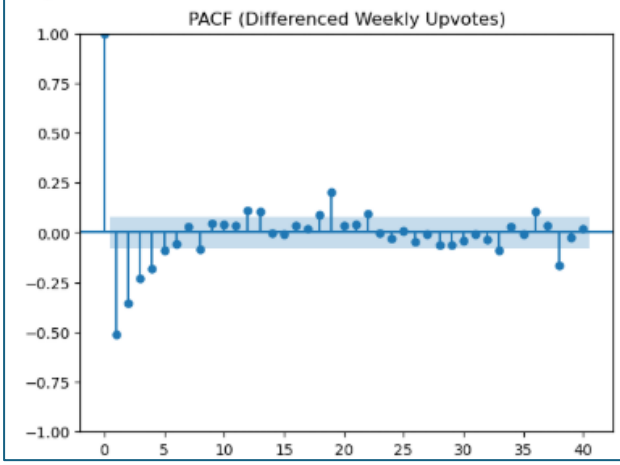


Figure 4.39: ACF and PACF of differenced series

Based on these plots, several candidate models were compared using Akaike Information Criterion (AIC), and ARIMA(1,1,2) was selected as the final model because it produced the lowest AIC value.

```
# store results
arima_results = {}

# candidate orders (based on ACF/PACF)
orders = [(1,1,1), (2,1,1), (1,1,2)]

for order in orders:
    print(f"Fitting ARIMA{order}...")

    model = ARIMA(ts_weekly, order=order)
    fitted = model.fit()

    arima_results[order] = fitted.aic
    print(f"AIC for ARIMA{order}: {fitted.aic:.2f}\n")

best_order = min(arima_results, key=arima_results.get)
print("Best ARIMA order (lowest AIC):", best_order, "AIC:", arima_results[best_order])
```

```
Fitting ARIMA(1, 1, 1)...
AIC for ARIMA(1, 1, 1): 16469.12

Fitting ARIMA(2, 1, 1)...
AIC for ARIMA(2, 1, 1): 16460.17

Fitting ARIMA(1, 1, 2)...
AIC for ARIMA(1, 1, 2): 16420.45

Best ARIMA order (lowest AIC): (1, 1, 2) AIC: 16420.450585196624
```

Figure 4.40: AIC comparison of candidate ARIMA models

Finally, the fitted ARIMA model was used to forecast weekly Reddit upvotes for 12 future periods.

```
# use best order from AIC search
final_fit = ARIMA(ts_weekly, order=(1,1,2)).fit()

print("Final ARIMA model fitted.")
```

```
Final ARIMA model fitted.
```

Figure 4.41: Final ARIMA Model Fitting Based on AIC Selection

The forecast plot in **Figure 4.42** shows the historical series, projected values, and confidence interval.

```python
# Generate 12-step ahead forecast from the fitted ARIMA model
forecast_res = final_fit.get_forecast(steps=12)

# Extract predicted mean values (the forecast line)
forecast_mean = forecast_res.predicted_mean

# Extract confidence interval bounds
conf_int = forecast_res.conf_int()

# Create plot
plt.figure(figsize=(10,5))

# Plot historical weekly upvotes
plt.plot(ts_weekly, label="Historical")

# Plot forecasted values
plt.plot(forecast_mean.index, forecast_mean, label="Forecast", color="red")

# Plot shaded confidence interval area
plt.fill_between(
    conf_int.index,
    conf_int.iloc[:, 0],  # Lower bound
    conf_int.iloc[:, 1],  # upper bound
    color="pink",
    alpha=0.3,
    label="Confidence Interval"
)

# Add titles and labels
plt.title("ARIMA Forecast of Weekly Reddit Upvotes")
plt.xlabel("Date")
plt.ylabel("Total Upvotes")

# Show legend and display plot
plt.legend()
plt.show()
```

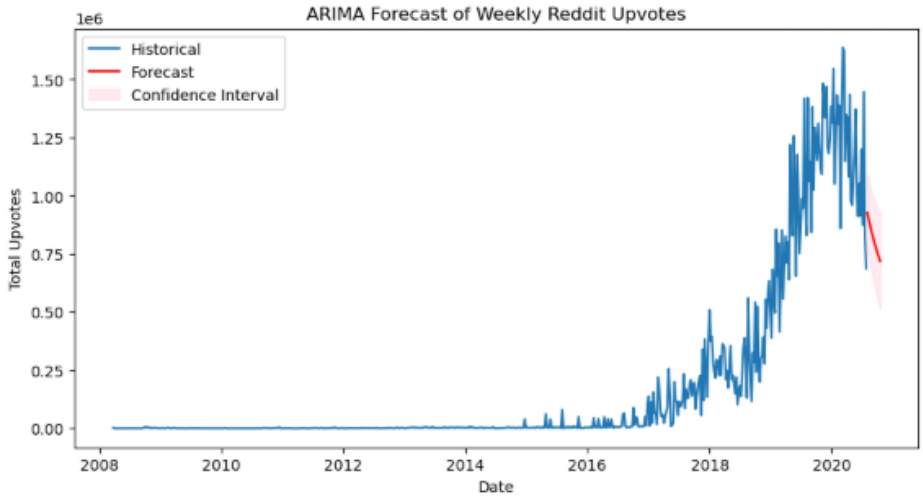


Figure 4.42: ARIMA Forecast of Weekly Reddit Upvotes with Confidence

The results suggest that although the ARIMA model captures the general short-term direction of the time series, forecasting remains uncertain. The historical data exhibits substantial fluctuations and sudden spikes in engagement, making precise prediction difficult. The widening confidence interval further indicates increasing uncertainty in the predicted values.

4.4.4 Subreddit Community Network Analysis (RQ4)

To investigate how subreddit community structure relates to viral engagement, a network-based analysis was implemented. The aim was to represent subreddits as nodes within a similarity network and examine whether their structural positions were associated with higher viral post rates.

4.4.4.1 Subreddit Document Construction

Subreddit-level documents were created by aggregating cleaned post text within each subreddit. Missing values were replaced with empty strings, and all posts were concatenated into a single document per subreddit. This produced 30 aggregated documents representing the overall language used in each community.

```
# select the cleaned text column used for analysis
text_col = "text_clean"

# ensure there are no missing text values
df[text_col] = df[text_col].fillna("").astype(str)

# combine all posts from each subreddit into one text document
sub_docs = df.groupby("Subreddit")[text_col].apply(lambda x: " ".join(x)).reset_index()

# rename columns
sub_docs.columns = ["Subreddit", "sub_text"]

# check number of subreddit documents created
print("Number of subreddits:", sub_docs.shape[0])
print(sub_docs.head())
```

```
Number of subreddits: 30
           Subreddit                                           sub_text
0     AdviceForTeens  make your flair your name or what you want to ...
1            Anxiety  i think we can all relate. (found on ifunny -_...
2  ApplyingToCollege  [humor] we should start referring to uchicago ...
3            COVID19  "let's start with what we do not know" excelle...
4           Colombia  look who made to the top ten in the world beau...
```

Figure 4.43: Combined Subreddit Text Data

4.4.4.2 TF-IDF Representation of Subreddits

TF-IDF vectorisation was applied to convert subreddit documents into numerical form, representing each subreddit as a feature vector. Unlike earlier TF-IDF applied at the post level, this approach treats each subreddit as a single aggregated document, enabling similarity comparison between communities. The vectoriser used up to 5,000 features with stopword removal and a minimum document frequency threshold, producing a (30 × 5000) matrix

```
# initialise TF-IDF vectoriser
vectorizer = TfidfVectorizer(
    max_features=5000,
    stop_words="english",
    min_df=2
)

# convert subreddit text documents into TF-IDF feature matrix
sub_tfidf = vectorizer.fit_transform(sub_docs["sub_text"])

# check resulting matrix size
print("TF-IDF matrix shape:", sub_tfidf.shape)
```

```
TF-IDF matrix shape: (30, 5000)
```

Figure 4.44: TF-IDF Vectorisation of Subreddit Text Data

4.4.4.3 Subreddit Similarity Matrix

Cosine similarity was then computed between all subreddit TF-IDF vectors using the cosine_similarity function from Scikit-learn.

```
# calculate cosine similarity between subreddit TF-IDF vectors
similarity_matrix = cosine_similarity(sub_tfidf)
```

Figure 4.45: Cosine Similarity Computation Between Subreddit TF-IDF Vectors

This step measured the textual similarity between subreddit communities based on the language used in their posts.

The similarity scores were organised into a similarity matrix and converted into a Pandas DataFrame for easier interpretation, with subreddit names used as both row and column labels.

Subreddit	AdviceForTeens	Anxiety	ApplyingToCollege	COVID19	\
Subreddit					
AdviceForTeens	1.000	0.725	0.668	0.071	
Anxiety	0.725	1.000	0.555	0.076	
ApplyingToCollege	0.668	0.555	1.000	0.090	
COVID19	0.071	0.076	0.090	1.000	
Colombia	0.072	0.068	0.064	0.018	
Entrepreneur	0.564	0.529	0.534	0.105	
Feminism	0.375	0.316	0.307	0.083	
GradSchool	0.608	0.522	0.610	0.104	
NeutralPolitics	0.278	0.275	0.307	0.135	
SuicideWatch	0.844	0.753	0.634	0.080	
YouthRights	0.520	0.408	0.516	0.100	
YouthandGovernment	0.375	0.316	0.339	0.060	
astrology	0.355	0.347	0.295	0.061	
business	0.217	0.207	0.236	0.101	
careerguidance	0.666	0.617	0.583	0.094	
college	0.727	0.623	0.799	0.099	
computerscience	0.508	0.434	0.494	0.102	
dating	0.827	0.696	0.607	0.079	
depression_help	0.890	0.805	0.644	0.082	
engineering	0.404	0.378	0.386	0.089	
highschool	0.769	0.571	0.754	0.083	
jobs	0.612	0.564	0.549	0.094	
kidsrights	0.213	0.170	0.229	0.094	
mexico	0.013	0.013	0.013	0.005	
povertyfinance	0.653	0.611	0.604	0.097	
science	0.216	0.238	0.243	0.236	
studentaffairs	0.410	0.348	0.507	0.073	
teenagers	0.786	0.680	0.673	0.088	
whatsbotheringyou	0.943	0.787	0.695	0.083	
youthshouldknow	0.453	0.396	0.484	0.079	

Subreddit	Colombia	Entrepreneur	Feminism	GradSchool	\
Subreddit					
AdviceForTeens	0.072	0.564	0.375	0.608	
Anxiety	0.068	0.529	0.316	0.522	
ApplyingToCollege	0.064	0.534	0.307	0.610	
COVID19	0.018	0.105	0.083	0.104	
Colombia	1.000	0.071	0.045	0.058	
Entrepreneur	0.071	1.000	0.307	0.539	
Feminism	0.045	0.307	1.000	0.282	
GradSchool	0.058	0.539	0.282	1.000	
NeutralPolitics	0.054	0.375	0.269	0.325	
SuicideWatch	0.074	0.576	0.367	0.561	
YouthRights	0.055	0.429	0.377	0.469	
YouthandGovernment	0.042	0.334	0.198	0.356	
astrology	0.041	0.326	0.212	0.280	
business	0.051	0.445	0.191	0.253	
careerguidance	0.065	0.640	0.306	0.685	
college	0.068	0.574	0.320	0.689	
computerscience	0.051	0.502	0.244	0.585	
dating	0.073	0.596	0.431	0.557	
depression_help	0.073	0.597	0.362	0.601	
engineering	0.048	0.458	0.227	0.432	
highschool	0.063	0.502	0.313	0.642	
jobs	0.062	0.616	0.282	0.648	
kidsrights	0.034	0.206	0.295	0.214	
mexico	0.818	0.015	0.012	0.011	
povertyfinance	0.074	0.692	0.324	0.616	
science	0.039	0.306	0.304	0.274	
studentaffairs	0.043	0.383	0.205	0.666	
teenagers	0.078	0.588	0.395	0.566	
whatsbotheringyou	0.078	0.626	0.393	0.642	
youthshouldknow	0.048	0.442	0.292	0.434	

Subreddit	kidsrights	mexico	povertyfinance	science	\
Subreddit					
AdviceForTeens	0.213	0.013	0.653	0.216	
Anxiety	0.170	0.013	0.611	0.238	
ApplyingToCollege	0.229	0.013	0.604	0.243	
COVID19	0.094	0.005	0.097	0.236	
Colombia	0.034	0.818	0.074	0.039	
Entrepreneur	0.206	0.015	0.692	0.306	
Feminism	0.295	0.012	0.324	0.304	
GradSchool	0.214	0.011	0.616	0.274	
NeutralPolitics	0.283	0.022	0.354	0.306	
SuicideWatch	0.221	0.015	0.676	0.248	
YouthRights	0.562	0.012	0.469	0.329	
YouthandGovernment	0.194	0.009	0.336	0.196	
astrology	0.138	0.011	0.330	0.208	
business	0.192	0.018	0.386	0.264	
careerguidance	0.202	0.012	0.734	0.266	
college	0.212	0.013	0.664	0.282	
computerscience	0.164	0.011	0.475	0.245	
dating	0.182	0.014	0.645	0.245	
depression_help	0.209	0.013	0.694	0.250	
engineering	0.159	0.011	0.450	0.220	
highschool	0.254	0.012	0.577	0.248	
jobs	0.182	0.012	0.696	0.248	
kidsrights	1.000	0.012	0.242	0.353	
mexico	0.012	1.000	0.015	0.010	
povertyfinance	0.242	0.015	1.000	0.290	
science	0.353	0.010	0.290	1.000	
studentaffairs	0.180	0.008	0.450	0.211	
teenagers	0.241	0.020	0.670	0.247	
whatsbotheringyou	0.224	0.014	0.741	0.253	
youthshouldknow	0.297	0.009	0.505	0.314	

Subreddit	NeutralPolitics	SuicideWatch	...	highschool	jobs	\
Subreddit			...			
AdviceForTeens	0.278	0.844	...	0.769	0.612	
Anxiety	0.275	0.753	...	0.571	0.564	
ApplyingToCollege	0.307	0.634	...	0.754	0.549	
COVID19	0.135	0.080	...	0.083	0.094	
Colombia	0.054	0.074	...	0.063	0.062	
Entrepreneur	0.375	0.576	...	0.502	0.616	
Feminism	0.269	0.367	...	0.313	0.282	
GradSchool	0.325	0.561	...	0.642	0.648	
NeutralPolitics	1.000	0.283	...	0.271	0.316	
SuicideWatch	0.283	1.000	...	0.651	0.592	
YouthRights	0.384	0.491	...	0.594	0.399	
YouthandGovernment	0.311	0.358	...	0.366	0.303	
astrology	0.214	0.375	...	0.291	0.284	
business	0.334	0.235	...	0.228	0.321	
careerguidance	0.315	0.668	...	0.579	0.956	
college	0.312	0.683	...	0.760	0.616	
computerscience	0.282	0.466	...	0.497	0.507	
dating	0.294	0.784	...	0.618	0.589	
depression_help	0.284	0.924	...	0.679	0.630	
engineering	0.265	0.412	...	0.378	0.495	
highschool	0.271	0.651	...	1.000	0.526	
jobs	0.316	0.592	...	0.526	1.000	
kidsrights	0.283	0.221	...	0.254	0.182	
mexico	0.022	0.015	...	0.012	0.012	
povertyfinance	0.354	0.676	...	0.577	0.696	
science	0.306	0.248	...	0.248	0.248	
studentaffairs	0.263	0.373	...	0.468	0.553	
teenagers	0.321	0.764	...	0.720	0.549	
whatsbotheringyou	0.307	0.919	...	0.730	0.687	
youthshouldknow	0.243	0.454	...	0.467	0.423	

Subreddit	studentaffairs	teenagers	whatsbotheringyou	\
Subreddit				
AdviceForTeens	0.410	0.786	0.943	
Anxiety	0.348	0.680	0.787	
ApplyingToCollege	0.507	0.673	0.695	
COVID19	0.073	0.088	0.083	
Colombia	0.043	0.078	0.078	
Entrepreneur	0.383	0.588	0.626	
Feminism	0.205	0.395	0.393	
GradSchool	0.666	0.566	0.642	
NeutralPolitics	0.263	0.321	0.307	
SuicideWatch	0.373	0.764	0.919	
YouthRights	0.403	0.546	0.520	
YouthandGovernment	0.295	0.370	0.386	
astrology	0.198	0.345	0.393	
business	0.182	0.262	0.256	
careerguidance	0.571	0.593	0.751	
college	0.546	0.684	0.764	
computerscience	0.428	0.472	0.527	
dating	0.369	0.751	0.864	
depression_help	0.406	0.753	0.944	
engineering	0.318	0.425	0.453	
highschool	0.468	0.720	0.730	
jobs	0.553	0.549	0.687	
kidsrights	0.180	0.241	0.224	
mexico	0.008	0.020	0.014	
povertyfinance	0.450	0.670	0.741	
science	0.211	0.247	0.253	
studentaffairs	1.000	0.372	0.437	
teenagers	0.372	1.000	0.803	
whatsbotheringyou	0.437	0.803	1.000	
youthshouldknow	0.350	0.460	0.492	

Subreddit	youthshouldknow
Subreddit	
AdviceForTeens	0.453
Anxiety	0.396
ApplyingToCollege	0.484
COVID19	0.079
Colombia	0.048
Entrepreneur	0.442
Feminism	0.292
GradSchool	0.434
NeutralPolitics	0.243
SuicideWatch	0.454
YouthRights	0.517
YouthandGovernment	0.333
astrology	0.234
business	0.213
careerguidance	0.461
college	0.521
computerscience	0.406
dating	0.444
depression_help	0.466
engineering	0.314
highschool	0.467
jobs	0.423
kidsrights	0.297
mexico	0.009
povertyfinance	0.505
science	0.314
studentaffairs	0.350
teenagers	0.460
whatsbotheringyou	0.492
youthshouldknow	1.000

Figure 4.46: Subreddit Content Similarity Matrix (Cosine Similarity)

4.4.4.4 Construction of Subreddit Network Graph

Using the similarity matrix, a network graph was constructed with the NetworkX library. Each subreddit was represented as a node within the network. Edges were created between subreddit pairs when their cosine similarity exceeded a threshold of 0.25, indicating sufficiently similar textual content. The similarity scores were used as edge weights, allowing stronger relationships between communities to be represented by higher weights.

```python
import networkx as nx

# set similarity threshold to decide when two subreddits are connected
threshold = 0.25

# create an empty graph
G = nx.Graph()

# add subreddit nodes to the graph
for sub in sim_df.index:
    G.add_node(sub)

# add edges between subreddits if similarity is above the threshold
for i, sub1 in enumerate(sim_df.index):
    for j, sub2 in enumerate(sim_df.columns):
        if j <= i:
            continue  # skip duplicate pairs and self-connections
        weight = sim_df.loc[sub1, sub2] # similarity score

         # create edge if similarity is strong enough
        if weight >= threshold:
            G.add_edge(sub1, sub2, weight=float(weight))

# display graph summary in a more organised way
print("\n--- Subreddit Network Summary ---")
print(f"Total nodes (subreddits): {G.number_of_nodes()}") # number of subreddits
print(f"Total edges (connections): {G.number_of_edges()}") #number of connections

print("\nExample edges with similarity weights:")
for edge in list(G.edges(data=True))[:10]:
    sub1, sub2, data = edge
    print(f"{sub1}  <-->  {sub2}   | similarity = {data['weight']:.3f}")
```

```
--- Subreddit Network Summary ---
Total nodes (subreddits): 30
Total edges (connections): 292

Example edges with similarity weights:
AdviceForTeens  <-->  Anxiety   | similarity = 0.725
AdviceForTeens  <-->  ApplyingToCollege   | similarity = 0.668
AdviceForTeens  <-->  Entrepreneur   | similarity = 0.564
AdviceForTeens  <-->  Feminism   | similarity = 0.375
AdviceForTeens  <-->  GradSchool   | similarity = 0.608
AdviceForTeens  <-->  NeutralPolitics   | similarity = 0.278
AdviceForTeens  <-->  SuicideWatch   | similarity = 0.844
AdviceForTeens  <-->  YouthRights   | similarity = 0.520
AdviceForTeens  <-->  YouthandGovernment   | similarity = 0.375
AdviceForTeens  <-->  astrology   | similarity = 0.355
```

Figure 4.47: Subreddit Network Summary

4.4.4.5 Network Centrality Calculation and Integration

To analyse community structure, centrality metrics including degree centrality, betweenness centrality, and weighted node strength were computed to capture the connectivity and influence of each subreddit.

```python
# calculate degree centrality for each subreddit
degree_centrality = nx.degree_centrality(G)

# calculate betweenness centrality (how important a node is as a bridge)
betweenness_centrality = nx.betweenness_centrality(G)

# calculate weighted degree (strength of connections)
strength = dict(G.degree(weight="weight"))

# combine all centrality measures into one dataframe
centrality_df = pd.DataFrame({
    "Subreddit": list(degree_centrality.keys()),
    "degree_centrality": list(degree_centrality.values()),
    "betweenness_centrality": list(betweenness_centrality.values()),
    "weighted_strength": [strength[node] for node in degree_centrality.keys()]
})

# sort subreddits by strongest connections
centrality_df = centrality_df.sort_values("weighted_strength", ascending=False)

print("Most central subreddits in the similarity network:")
print(centrality_df.head(10))
```

```
Most central subreddits in the similarity network:
           Subreddit  degree_centrality  betweenness_centrality  \
28  whatsbotheringyou           0.862069                0.008740
18    depression_help           0.793103                0.001656
15            college           0.862069                0.008740
0      AdviceForTeens           0.793103                0.001656
14     careerguidance           0.862069                0.008740
24     povertyfinance           0.862069                0.008740
9        SuicideWatch           0.793103                0.001656
27          teenagers           0.827586                0.005171
17             dating           0.793103                0.001656
20         highschool           0.827586                0.011754

    weighted_strength
28          15.314065
18          14.086231
15          14.008536
0           13.957849
14          13.833335
24          13.640381
9           13.608072
27          13.549383
17          13.238071
20          12.976273
```

Figure 4.48: Subreddit Centrality Results

The network was visualised using NetworkX with a spring layout, where nodes represent subreddits and edges represent similarity relationships (**Figure 4.49**).

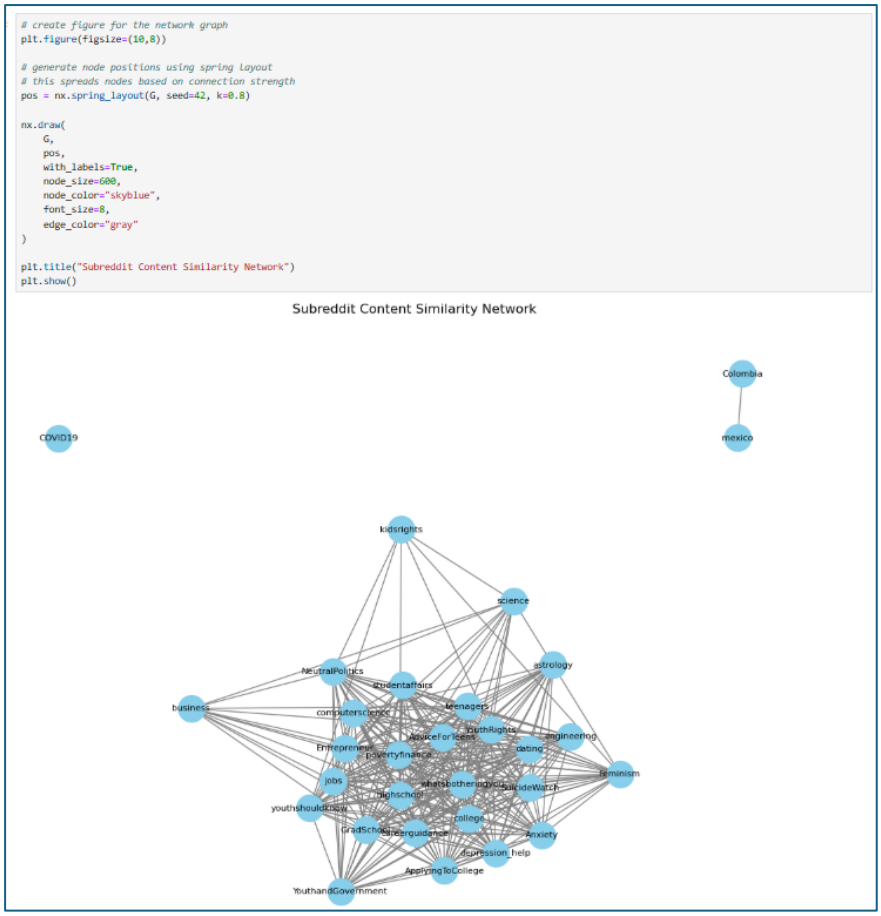


Figure 4.49: Subreddit Content Similarity Network Visualisation

Finally, the centrality metrics were merged with subreddit-level engagement statistics. Viral rate was calculated as the proportion of posts labelled as viral within each subreddit, while post count represents the number of posts per community. This integration allows examination of whether subreddit network position is associated with viral engagement.

```python
# calculate subreddit-level statistics
# viral_rate = proportion of viral posts in each subreddit
# post_count = number of posts per subreddit
sub_stats = df.groupby("Subreddit").agg({
    "is_viral": "mean",
    "Subreddit": "count"
}).rename(columns={"Subreddit": "post_count"})

# merge network centrality metrics with subreddit statistics
# this combines network position with virality information
network_stats = centrality_df.merge(
    sub_stats.reset_index(),
    on="Subreddit",
    how="left"
)

# rename column
network_stats = network_stats.rename(columns={"is_viral": "viral_rate"})

# reorder columns so it matches the evaluation table format
cols = [
    "Subreddit",
    "viral_rate",
    "post_count",
    "degree_centrality",
    "betweenness_centrality",
    "weighted_strength"
]

# display clean formatted table
from IPython.display import display

print("Merged rows:", network_stats.shape[0])
display(network_stats[cols].head(10))
```

Merged rows: 30

	Subreddit	viral_rate	post_count	degree_centrality	betweenness_centrality	weighted_strength
0	whatsbotheringyou	0.000000	996	0.862069	0.008740	15.314065
1	depression_help	0.000000	1000	0.793103	0.001656	14.086231
2	college	0.041037	926	0.862069	0.008740	14.008536
3	AdviceForTeens	0.000000	818	0.793103	0.001656	13.957849
4	careerguidance	0.000000	1000	0.862069	0.008740	13.833335
5	povertyfinance	0.467677	990	0.862069	0.008740	13.640381
6	SuicideWatch	0.003043	986	0.793103	0.001656	13.608072
7	teenagers	1.000000	957	0.827586	0.005171	13.549383
8	dating	0.008056	993	0.793103	0.001656	13.238071
9	highschool	0.000000	998	0.827586	0.011754	12.976273

Figure 4.50: Integration of Subreddit Centrality Metrics with Engagement

5 EVALUATION (2526 words)

5.1 Evaluation of Viral Post Classification (RQ1)

This section evaluates the classification models developed to address Research Question 1, which examines whether Reddit posts can be predicted as viral or non-viral. Model performance is assessed using accuracy, precision, recall, F1-score, and ROC-AUC. These metrics provide an assessment of classification performance under the imbalanced distribution of viral and non-viral posts. The evaluation compares Logistic Regression, Random Forest, and XGBoost classifiers using the held-out test dataset.

5.1.1 Classification Performance Comparison

The overall performance of the evaluated classifiers is summarised in **Figure 5.1** and illustrated in **Figure 5.2.**

Model Comparison Table (Classification)

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
0	Logistic Regression (Baseline)	0.9460	0.6867	0.8466	0.7583	0.9750
1	Logistic Regression (CV)	0.9445	0.6841	0.8289	0.7494	0.9648
2	Random Forest (Baseline)	0.9661	0.9947	0.6649	0.7970	0.9647
3	Random Forest (GridSearch)	0.9661	0.9947	0.6649	0.7970	0.9647
4	XGBoost (Baseline)	0.9458	0.6641	0.9277	0.7741	0.9872
5	XGBoost (GridSearch)	0.9652	0.7946	0.8801	0.8351	0.9867

Figure 5.1: Performance Comparison of Classification Models

Across all models, classification accuracy ranged between 0.9445 and 0.9661, indicating that the models were effective at distinguishing between viral and non-viral posts. However, accuracy alone is insufficient because viral posts represent a minority class within the dataset. Therefore, additional metrics such as recall and F1-score were considered when comparing models.

Among the evaluated classifiers, the tuned XGBoost model achieved the strongest performance, obtaining the highest F1-score (0.8351), recall (0.8801), and ROC-AUC (0.9867). These results indicate that XGBoost was more effective at identifying viral posts while maintaining strong classification capability. The improved performance of XGBoost can be attributed to its gradient boosting framework, which combines multiple decision trees to capture nonlinear relationships between textual content, posting behaviour, and community-level activity features.

The Random Forest classifier achieved the highest precision (0.9947), indicating that posts predicted as viral were almost always correct. However, its recall was substantially lower (0.6649), suggesting that the model failed to identify many viral posts. This behaviour indicates that Random Forest prioritised precision over detecting viral posts.

Logistic Regression served as a baseline model, achieving an accuracy of 0.9460 and an F1-score of 0.7583. Although its performance was lower than the ensemble models, it produced relatively balanced precision and recall.

Overall, these results indicate that ensemble-based models outperform the linear baseline, suggesting that predicting Reddit post virality involves nonlinear interactions between multiple features.

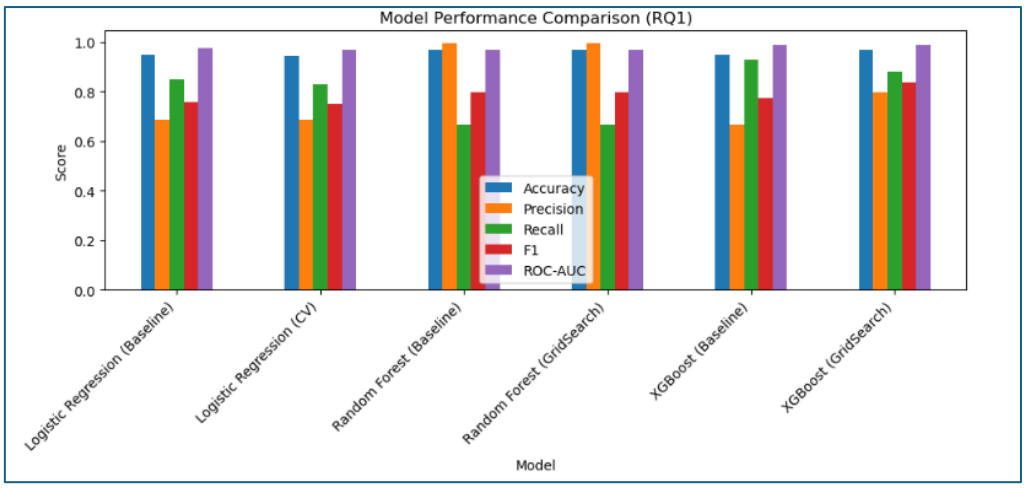


Figure 5.2: Visual Comparison of Classification Models Across Evaluation

5.1.2 ROC and Precision–Recall Analysis

The ROC and Precision–Recall (PR) curves shown in **Figure 5.3** further evaluate the classification capability of the models across different decision thresholds.

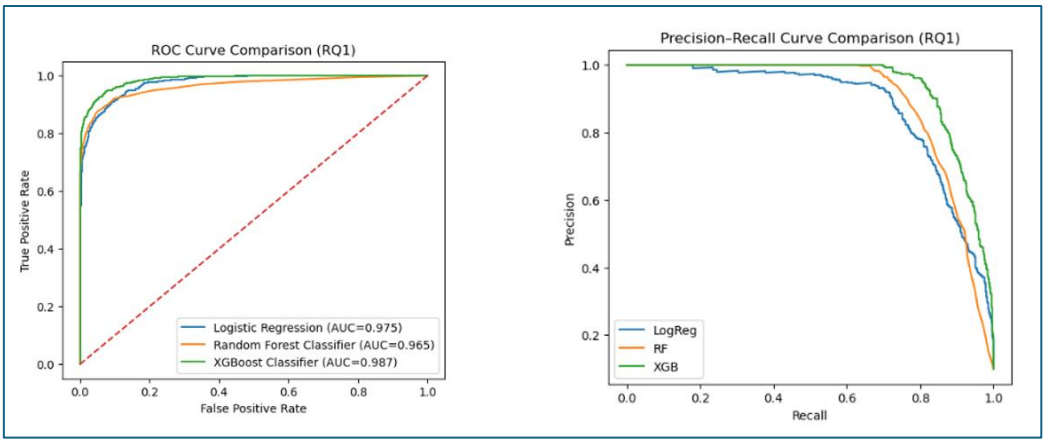


Figure 5.3: ROC and Precision–Recall Curve Comparison

All classifiers achieved strong discriminative performance, with ROC–AUC values exceeding 0.96, indicating that the models effectively separate viral from non-viral posts. Among the evaluated models, XGBoost achieved the highest ROC–AUC score of approximately 0.987, demonstrating the strongest class separability. Logistic Regression achieved a ROC–AUC of 0.975, while Random Forest produced a slightly lower ROC–AUC of 0.965.

Given the imbalanced dataset, the Precision–Recall curve provides an additional perspective on classification performance. As illustrated in **Figure 5.3**, the PR curve for XGBoost consistently remains above those of Logistic Regression and Random Forest, indicating higher precision across a wider range of recall values and improved detection of viral posts.

Overall, these results reinforce the earlier model comparison, confirming that XGBoost provides the most reliable classification performance for detecting viral posts.

5.1.4 Confusion Matrix Interpretation

The confusion matrices shown in **Figure 5.4** provide further insight into the prediction behaviour of each classifier.

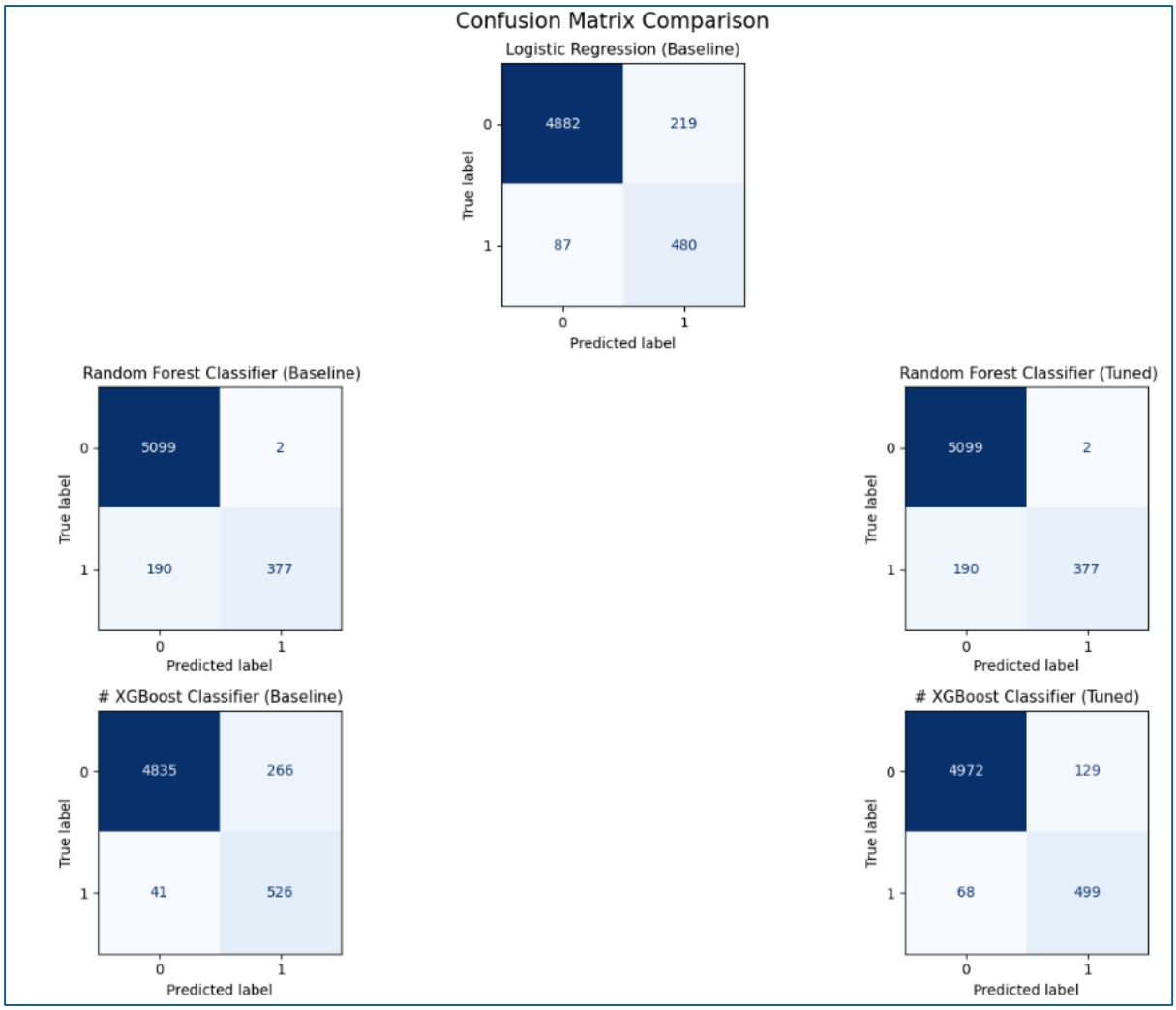


Figure 5.4: Confusion Matrix Comparison of Classification Models

The Logistic Regression model correctly classified 4,882 non-viral posts and 480 viral posts, indicating reasonable detection capability while maintaining balanced predictions.

The Random Forest classifier produced extremely few false positives, correctly identifying 5,099 non-viral posts, but misclassified a larger number of viral posts. This explains the model's very high precision but comparatively lower recall.

In contrast, the tuned XGBoost classifier achieved improved detection of viral posts, correctly identifying 499 viral posts while reducing the number of false negatives compared with the baseline model. This improved sensitivity suggests that XGBoost is better able to capture patterns associated with viral engagement.

Overall, the confusion matrix analysis confirms that XGBoost achieves the most balanced trade-off between identifying viral posts and avoiding incorrect classifications.

5.1.5 Cross-Validation Stability

To assess model robustness, Stratified 5-Fold Cross-Validation was applied to the Logistic Regression classifier on the training dataset. The results indicate consistent performance across validation folds, suggesting that the model generalises well and is unlikely to be overfitting to the training data.

A detailed summary of the cross-validation results is provided in **Appendix G.**

5.2 Evaluation of Upvote Prediction (RQ2)

This section evaluates the regression models developed to address Research Question 2, which investigates whether Reddit engagement levels, measured by upvotes, can be predicted using textual, temporal, and community-level features. Model performance is assessed using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2, which together measure prediction accuracy, error magnitude, and explanatory power.

5.2.1 Model Comparison

The overall performance of the regression models is summarised in **Figure 5.5** and illustrated in **Figure 5.6**.

Regression Model Comparison

	Model	MAE	RMSE	R2
0	Linear Regression (Baseline)	1.5388	1.9968	0.4808
1	Linear Regression (CV)	1.6207	2.1190	0.4086
2	RF Regressor (Baseline)	0.4280	0.6018	0.9528
3	RF Regressor (GridSearch)	0.4288	0.5988	0.9533
4	XGB Regressor (Baseline)	0.4483	0.6125	0.9511
5	XGB Regressor (GridSearch)	0.4453	0.6092	0.9517

Figure 5.5: Performance Comparison of Regression Models

Linear Regression produced the weakest predictive performance, with an MAE of 1.5388, RMSE of 1.9968, and an R^2 score of 0.4808. These results indicate that the linear model explains less than half of the variance in Reddit engagement levels. This suggests that the relationship between the input features and engagement outcomes is highly complex and cannot be adequately represented by a simple linear model.

In contrast, the ensemble-based models significantly outperformed the linear baseline. The Random Forest Regressor achieved the strongest performance, with an RMSE of 0.5988, MAE of 0.4288, and an R^2 score of 0.9533 after hyperparameter tuning. The XGBoost Regressor achieved similar performance, with an RMSE of 0.6092 and an R^2 score of 0.9517.

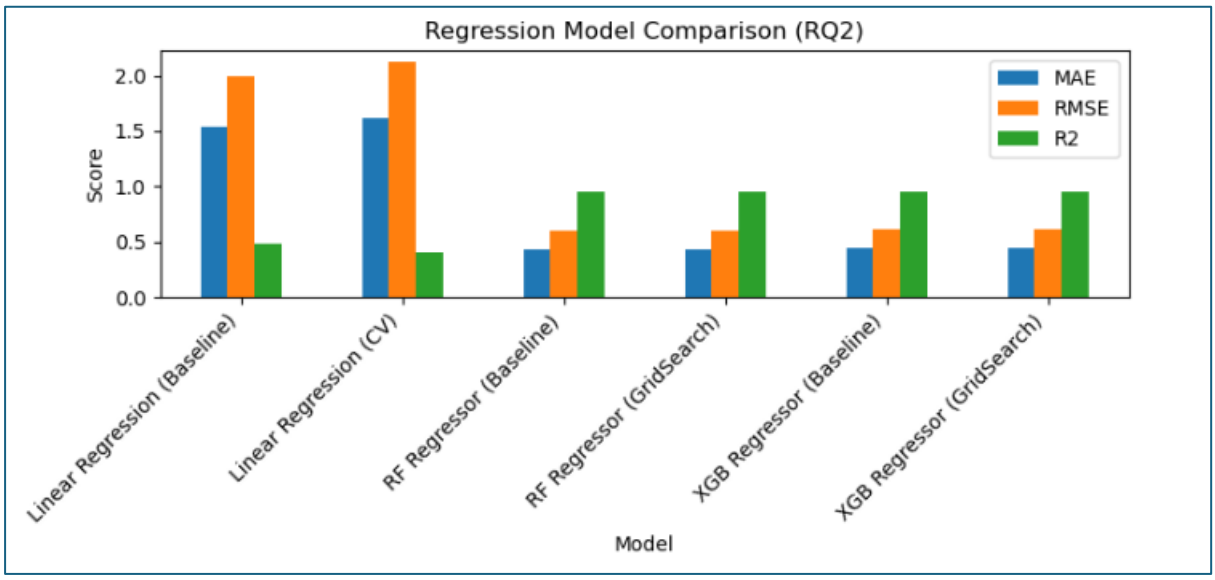


Figure 5.6: Visual Comparison of Regression Models Across Evaluation

These results indicate that ensemble models are substantially more effective at predicting Reddit engagement. Tree-based ensemble methods are able to capture nonlinear feature interactions and complex relationships between textual content, posting behaviour, and subreddit activity. This capability allows them to model engagement dynamics more accurately than traditional linear regression approaches.

5.2.2 Actual vs Predicted Scatter Plot

The Actual vs Predicted scatter plots shown in **Figure 5.7** provide further insight into model performance by comparing predicted values with the true log-transformed upvote values.

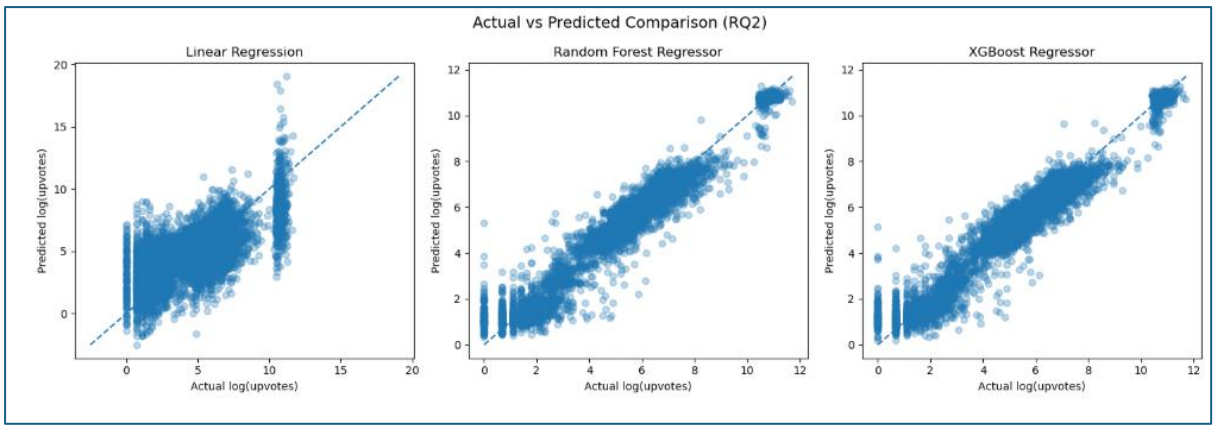


Table 5.7: Actual vs Predicted Values Comparison for Regression Models

For Linear Regression, the predictions are widely dispersed around the diagonal reference line, indicating weaker predictive accuracy. The model struggles to capture posts with higher engagement levels and produces large deviations between predicted and actual values.

In contrast, the Random Forest and XGBoost regressors show much stronger alignment along the diagonal, indicating that predicted values closely follow the actual engagement values. This demonstrates that the ensemble models are able to capture the relationship between the input features and Reddit engagement more effectively.

The improved alignment also indicates that the ensemble models are better able to model the variability in engagement levels across posts.

5.2.3 Residual Plot Comparison

The residual plots shown in **Figure 5.8** illustrate the distribution of prediction errors relative to predicted values.

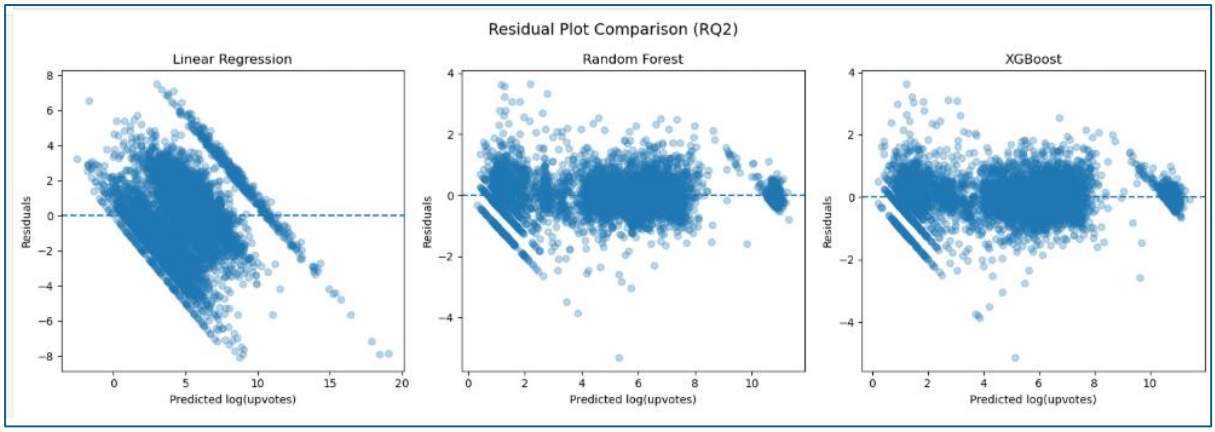


Table 5.8: Residual Plot of Regression Models

For the Linear Regression model, the residuals form a clear structured pattern rather than appearing randomly scattered around zero. This pattern indicates systematic modelling errors and suggests that important nonlinear relationships remain unmodelled.

In contrast, the Random Forest and XGBoost models show residuals that are more evenly distributed around zero, indicating that these models provide a better overall fit to the data. The absence of strong patterns in the residual plots suggests that ensemble models are more effective at capturing the underlying engagement dynamics.

5.2.4 Additional Model Diagnostic Analysis

Additional diagnostic analyses were conducted to examine model behaviour. Cross-validation results show that Linear Regression produced consistent RMSE values across folds, indicating stable performance. Residual analysis shows wider errors for Linear Regression, while Random Forest and XGBoost errors are more concentrated around zero. Detailed results are provided in **Appendix H**.

5.3 Time-Series Analysis of Reddit Engagement (RQ3)

This section evaluates the time-series modelling approach developed to address Research Question 3, which investigates how Reddit engagement evolves over time and whether time-series modelling can capture these dynamics. Weekly total upvotes were aggregated to form a continuous time series, and an ARIMA(1,1,2) model was selected based on the lowest Akaike Information Criterion (AIC) among candidate models.

5.3.1 Forecast Accuracy

Forecast accuracy was evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) on a holdout test period. The ARIMA model achieved an MAE of 209,914.65 and an RMSE of 233,922.18.

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
from statsmodels.tsa.arima.model import ARIMA

# holdout last 12 weeks
train = ts_weekly.iloc[:-12]
test  = ts_weekly.iloc[-12:]

model = ARIMA(train, order=(1,1,2)).fit()
pred = model.forecast(steps=12)

mae = mean_absolute_error(test, pred)
rmse = np.sqrt(mean_squared_error(test, pred))

print("RQ3 Forecast Evaluation")
print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))
```

```
RQ3 Forecast Evaluation
MAE: 209914.65
RMSE: 233922.18
```

Table 5.9: ARIMA Forecast Evaluation

These error values indicate that while the model is able to approximate the general level of Reddit engagement, prediction errors remain relatively large. This reflects the highly volatile nature of online engagement, where sudden spikes in attention frequently occur due to viral events. Because ARIMA models rely on historical temporal patterns, they are able to capture overall temporal trends but may struggle to accurately predict abrupt fluctuations driven by external social factors.

Despite these limitations, the model provides useful insights into the broader engagement dynamics observed across the platform.

5.3.2 Forecast vs Actual Comparison

Figure 5.10 compares the predicted and actual weekly Reddit engagement values during the evaluation period.

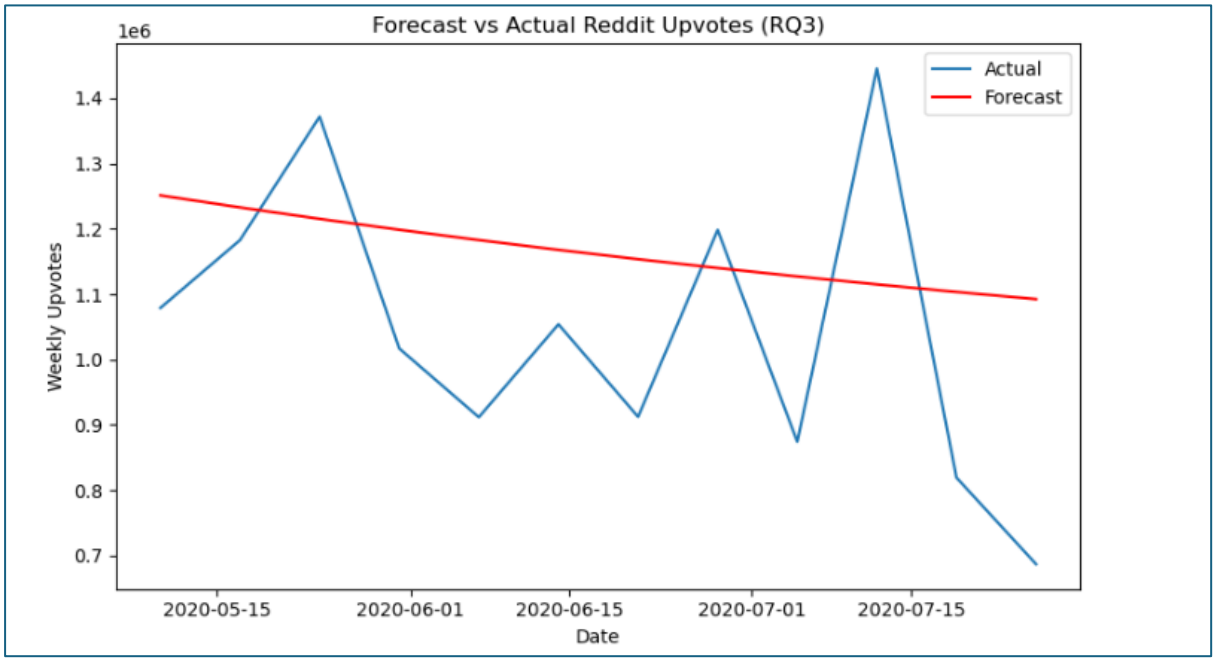


Table 5.10: Comparison of ARIMA Forecast and Actual Weekly Reddit Upvotes

The forecast generally follows the overall direction of the observed time series, suggesting that the ARIMA model captures the underlying temporal structure of engagement.

However, deviations are visible during periods of sharp increases or decreases in engagement. These discrepancies highlight the difficulty of forecasting highly dynamic social media behaviour, where engagement levels may change rapidly due to trending discussions or viral content. Consequently, while the model performs reasonably well in capturing overall trends, its ability to predict sudden engagement spikes remains limited.

5.3.3 ARIMA Forecast Behaviour

As shown in **Figure 4.42 (Section 4.4.2),** the ARIMA forecast illustrates the predicted trajectory of weekly Reddit engagement together with the associated confidence interval. The forecast indicates a gradual downward trend during the prediction period, while the confidence interval widens as the forecast horizon increases. This widening interval reflects increasing uncertainty in longer-term predictions, which is a common characteristic of time-series forecasting models.

The relatively smooth forecast line also illustrates a limitation of ARIMA modelling. While the model captures the general trajectory of engagement, it tends to smooth out short-term fluctuations that frequently occur in online platforms. As a result, extreme engagement events are not fully represented in the forecasted values.

Additional diagnostic analyses of forecast errors and residual autocorrelation are provided in **Appendix I** to further evaluate model behaviour and confirm that no significant temporal structure remains in the residuals.

5.4 Evaluation of Subreddit Community Network Analysis(RQ4)

This section analyses the structural relationships between Reddit communities using a subreddit similarity network. The objective is to examine whether a subreddit's position within the network is associated with higher levels of viral engagement.

5.4.1 Subreddit Similarity Network Structure

The subreddit similarity network constructed in **Figure 4.49 (Section 4.4.4.5)** is used to analyse structural relationships between communities.

As shown in Figure A , the network reveals a large central cluster where many communities are densely interconnected. Subreddits such as college, anxiety, teenagers, and advice-related communities appear within this cluster, indicating that they share overlapping themes and discussion patterns. These communities often focus on similar social or personal topics, which results in stronger textual similarity.

Several communities also appear relatively isolated from the main cluster. For example, COVID19 appears as a separate node with limited connections, suggesting that its discussion topics differ significantly from the broader network. A small pair cluster consisting of Mexico and Colombia is also visible, indicating localised similarity between these communities but weaker similarity with other subreddits.

Overall, the network structure suggests that Reddit communities tend to form clusters based on shared discussion themes and content similarity.

5.4.2 Centrality and Viral Engagement

To evaluate whether structurally influential communities produce more viral posts, the relationship between network centrality metrics and subreddit viral rates was analysed. Two centrality measures were examined: degree centrality and betweenness centrality.

Figure 5.11 shows the relationship between degree centrality and subreddit viral rate. Degree centrality measures the number of connections a subreddit has within the similarity network. If highly connected communities were more likely to generate viral posts, a positive relationship would be expected.

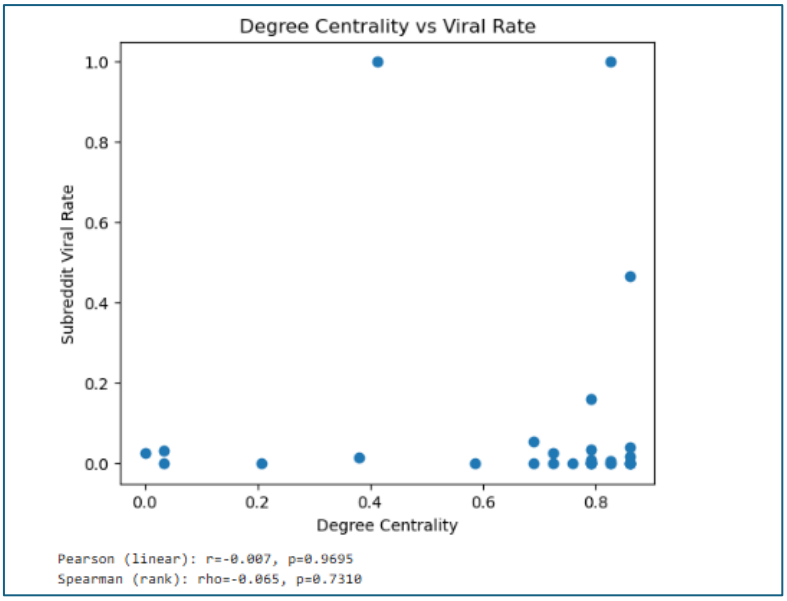


Figure 5.11: Scatter Plot of Degree Centrality vs Viral Rate

However, the scatter plot shows no clear pattern between degree centrality and viral engagement. Most subreddits exhibit relatively low viral rates regardless of their number of network connections. This indicates that communities with more similarity connections do not necessarily produce more viral content.

A similar pattern is observed when examining betweenness centrality. Betweenness centrality measures how often a subreddit lies on the shortest paths between other communities, indicating whether it acts as a bridge between discussion topics.

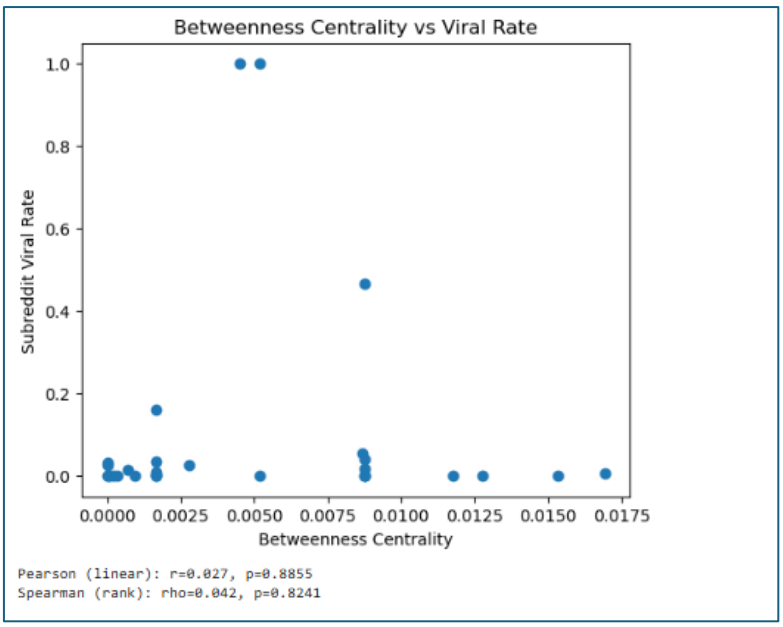


Figure 5.12: Scatter Plot of Between Centrality vs Viral Rate

As shown in Figure 5.12, no strong relationship between betweenness centrality and viral rate is visible. Communities that occupy bridging positions within the network do not consistently demonstrate higher viral engagement.

This observation is further supported by correlation analysis (**Figure 5.13**), which shows near-zero relationships between viral rate and both degree centrality ($r \approx -0.01$) and betweenness centrality ($r \approx 0.03$), indicating that network position has minimal influence on viral engagement.

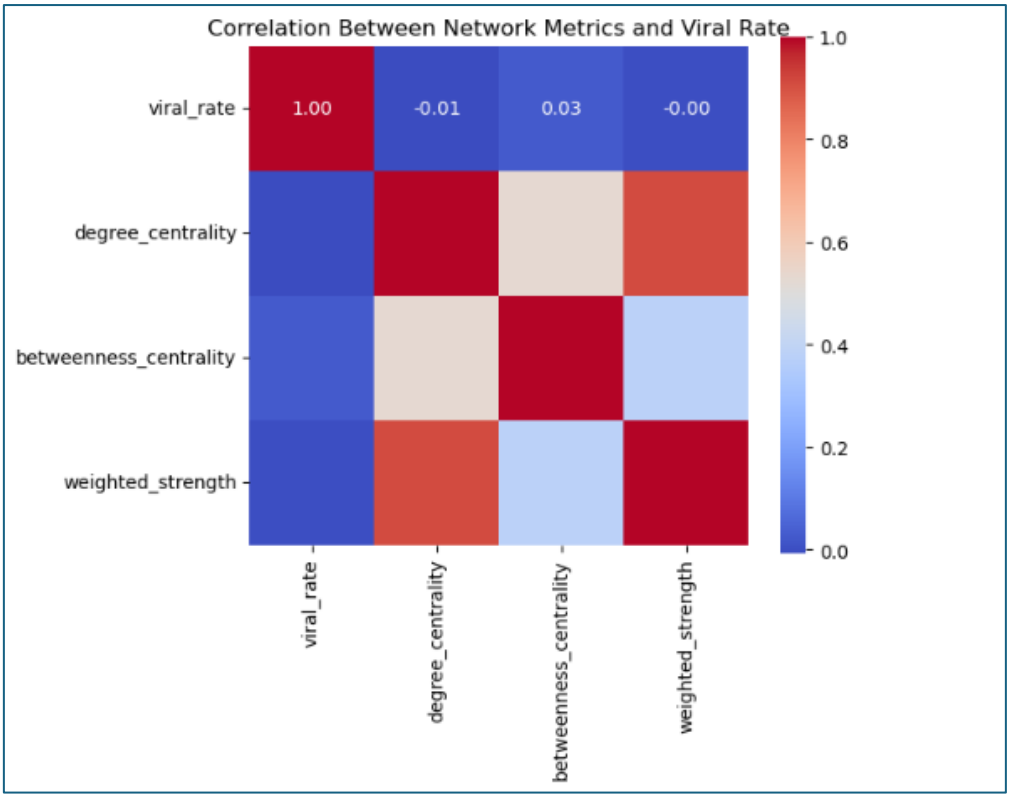


Figure 5.13: Correlation Heatmap of Network Centrality Metrics and Viral

Additional network analysis results, including detailed correlation tables and supplementary statistical tests, are provided in **Appendix J**.

5.5 Limitations and Potential Improvements

Despite the strong predictive performance achieved by the models, several limitations should be acknowledged together with opportunities for improvement.

First, the dataset used in this study was obtained from a Kaggle snapshot of Reddit posts rather than the live Reddit API. Consequently, the data represents a static view of platform activity and does not capture real-time dynamics such as evolving discussions or emerging viral trends. Future work could incorporate continuously updated datasets or API-based data collection to better capture temporal engagement patterns.

Second, the dataset does not include detailed user interaction structures such as reply trees or user-to-user networks. These interaction patterns influence how information spreads within online communities. Future research could incorporate interaction graphs or diffusion networks to better understand how viral content propagates across Reddit.

Third, viral posts were defined using the 90th percentile of the upvote distribution. While this provides a consistent threshold, engagement levels vary across subreddits. Future studies

could explore community-specific thresholds that better reflect differences between communities.

Finally, textual features relied on TF-IDF vectorisation and sentiment analysis, which may not fully capture contextual meaning in social media text. In addition, the ARIMA time-series model assumes relatively stable temporal patterns and may struggle to capture sudden spikes in engagement. Future research could explore transformer-based language models and more flexible machine learning time-series approaches.

5.6 Summary of Evaluation

This chapter evaluated the proposed models across classification, regression, time-series modelling, and network analysis to address the research questions of the study.

For RQ1, machine learning models successfully predicted whether Reddit posts become viral. The tuned XGBoost classifier achieved the strongest performance, demonstrating the effectiveness of ensemble learning methods for modelling complex relationships between textual, temporal, and community-level features.

For RQ2, the regression analysis showed that Reddit engagement levels can be predicted with high accuracy. Random Forest and XGBoost regression models significantly outperformed the linear regression baseline, indicating that engagement dynamics involve nonlinear feature interactions.

For RQ3, time-series analysis revealed that Reddit engagement follows identifiable temporal patterns. Although the ARIMA model captured overall engagement trends, predicting sudden spikes remains challenging due to the volatile nature of online attention.

For RQ4, the subreddit similarity network showed that communities cluster around shared discussion topics. However, network centrality metrics showed little relationship with viral engagement, suggesting that virality is driven more strongly by post content and user behaviour than by structural community position.

Overall, the study demonstrates that combining textual, temporal, and community-level features with machine learning provides a useful framework for analysing and predicting Reddit virality.

6 CONCLUSION (286 WORDS)

This project aimed to analyse and predict Reddit post virality using a multi-method, data-driven framework that integrates textual, sentiment, temporal, and community-level features. By combining classification and regression approaches, the study provides both predictive insights and a deeper understanding of how engagement patterns emerge on Reddit.

The results demonstrate that machine learning models are effective in predicting viral outcomes. Ensemble methods, particularly XGBoost and Random Forest, achieved the strongest performance, outperforming linear baseline models in both classification and regression tasks. This indicates that virality is influenced by complex, non-linear interactions between multiple factors rather than simple relationships captured by linear models.

Beyond prediction, the project also highlights important behavioural insights. Temporal analysis shows that Reddit engagement follows identifiable patterns over time, although sudden spikes remain difficult to forecast accurately. Network analysis further reveals that while subreddit communities form clusters based on shared discussion themes, network position has minimal influence on viral engagement. Instead, content characteristics and user interaction appear to play a more significant role in determining visibility.

However, several limitations should be noted. The use of a static Kaggle dataset limits the ability to capture real-time engagement dynamics, and the absence of detailed user interaction data restricts deeper analysis of information diffusion. In addition, although Random Forest and XGBoost models performed well, the scope of hyperparameter tuning was constrained by computational limitations. Exploring a larger parameter space would significantly increase training time due to the high-dimensional feature set.

Future work could address these limitations by incorporating real-time Reddit API data, analysing user interaction networks, and applying more advanced optimisation techniques such as Bayesian tuning or deep learning models.

Overall, this project demonstrates that a multi-method approach is essential for understanding and predicting social media virality.

7 APPENDIX

Appendix A: Tools Used in Prior Research

Across the studies reviewed, researchers tend to rely on a similar set of tools to analyse social media data and build predictive models. Scikit-learn is widely used for implementing classification and regression algorithms such as Logistic Regression, Linear Regression, and Random Forest, while XGBoost is commonly applied for more advanced boosting-based models (Alonso-López et al., 2020; Kim & Hwang, 2025; Zhang et al., 2022). For analysing

network structures, tools such as NetworkX allow researchers to examine how information spreads through interconnected communities (Makow et al., 2021).

Sentiment in social media studies is often measured using tools such as VADER and TextBlob, which are designed to analyse short, informal online text (Kim & Hwang, 2025). In addition, textual data is frequently transformed into numerical features using vectorisation techniques such as TF-IDF, enabling machine-learning models to process large volumes of user-generated content.

For temporal analysis, ARIMA models are commonly used to analyse short-term engagement patterns and trends in social media activity (Pinto et al., 2013; Roy & Basu, 2020). Many studies also rely on platform APIs to collect posts, comments, and engagement data directly from social media platforms (Alonso-López et al., 2020; Zhang et al., 2022). Together, these tools form the core technical foundation across a wide range of virality and engagement-prediction research.

Appendix for project design

Appendix B: Project Development Plan

B.1 Project Timeline

The development of the project followed a structured timeline that organised the work into several sequential stages. The timeline was designed to ensure that sufficient time was allocated for data preparation, model development, analysis, and documentation.

Early stages of the project focused on obtaining and preparing the dataset, while later stages concentrated on building predictive models and analysing engagement patterns. The final phase involved evaluating the results and completing the written report.

This structured development process ensured that the analytical components of the system were implemented systematically and that sufficient time was available for experimentation and refinement.

B.2 Task Breakdown

The project development process consisted of several key stages. The first stage involved dataset collection, where Reddit post datasets were obtained from the publicly available Kaggle Reddit dataset and organised for analysis. This was followed by data cleaning, which included removing duplicate records, handling missing values, and standardising timestamps to ensure consistent data quality.

After the dataset was prepared, feature engineering was performed to generate predictive variables from the raw data. These features included textual indicators derived from post

content, sentiment scores computed using VADER, temporal variables describing posting time, and community-level attributes related to subreddit activity.

The next stage involved model development, where machine learning algorithms such as Logistic Regression, Random Forest, and XGBoost were implemented to perform both classification and regression tasks. Once the models were developed, model evaluation was conducted using appropriate performance metrics to assess the effectiveness of the different modelling approaches.

Finally, the report writing stage involved documenting the project methodology, experimental results, and analytical findings in the final dissertation.

B.3 Gantt Chart

The overall development schedule of the project is illustrated using a Gantt chart, which shows how the different development phases were distributed across the project timeline.

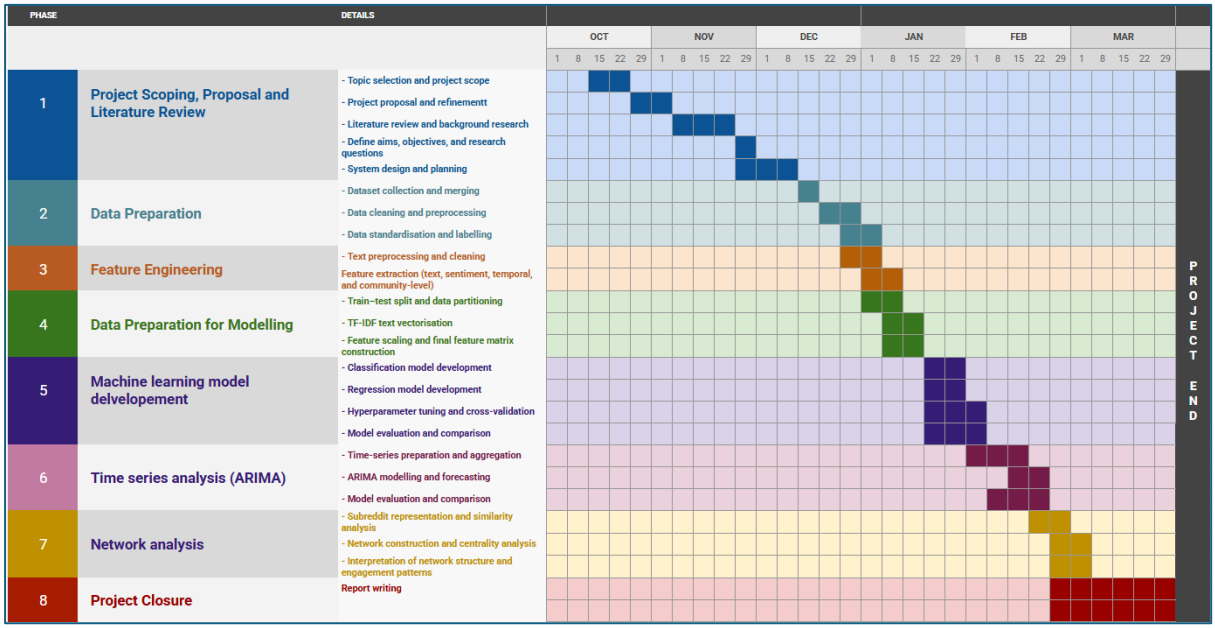


Figure B.1 Project development timeline represented using a Gantt chart.

The Gantt chart provides a visual overview of how tasks such as dataset preparation, feature engineering, modelling, evaluation, and report writing were scheduled throughout the duration of the project.

Appendix C – Data Cleaning and Preprocessing Details

Appendix C.1 Feature Selection and Removal of Irrelevant Variables

To ensure that the dataset contained only variables relevant to the modelling objectives, unnecessary columns were removed. These included unused attributes such as index columns, URLs, comment text, and duplicate or redundant fields.

The retained variables for analysis were:

- Post ID
- Title
- Body text
- Upvotes
- Creation date
- Subreddit
- Number of comments

These variables capture key aspects of Reddit engagement, including textual content (title and body), user interaction (upvotes and number of comments), temporal information (creation date), and community context (subreddit), all of which are relevant for predicting virality.

After filtering, the dataset contained 29,179 observations and 7 variables, forming the base dataset for subsequent preprocessing and analysis.

```python
# Drop unnecessary columns
df = df.drop(columns=[
    "Unnamed: 0",   # useless index column
    "URL",          # not used
    "Comments",     # comment text (not needed)
    "Flair",        # not used in this project
    "is_Original",  # duplicate
    "is_original"   # not needed for this coursework
], errors="ignore")

# Keep only relevant columns for the project
keep_cols = [
    "ID",
    "Title",
    "Body",
    "Upvotes",
    "creation_date",
    "Subreddit",
    "num_comments"
]

df = df[keep_cols]

print("Final columns used for modelling:")
print(df.columns)
print("Dataset shape (rows, columns):", df.shape)
df.head()
```

```
Final columns used for modelling:
Index(['ID', 'Title', 'Body', 'Upvotes', 'creation_date', 'Subreddit',
       'num_comments'],
      dtype='object')
Dataset shape (rows, columns): (29179, 7)
```

Figure C.1 Data Preprocessing and Final Variable Selection for Modelling

Appendix C.2 Data Type Conversion

To ensure consistency and compatibility with machine learning models, data types were standardised. Upvote counts and comment counts were converted to numeric format, while timestamps were transformed into datetime format. This step ensures that numerical computations and time-based feature extraction can be performed accurately in subsequent stages of analysis.

```
# Convert numeric columns
df["Upvotes"] = pd.to_numeric(df["Upvotes"], errors="coerce")
df["num_comments"] = pd.to_numeric(df["num_comments"], errors="coerce")

# Convert timestamp
df["creation_date"] = pd.to_datetime(df["creation_date"], errors="coerce")

print("Data types after conversion:")
print(df.dtypes)
```

```
Data types after conversion:
ID                       object
Title                    object
Body                     object
Upvotes                   int64
creation_date    datetime64[ns]
Subreddit                object
num_comments              int64
dtype: object
```

Figure C.2: Data Type Standardisation for Numerical and Temporal Variables

Appendix C.3 Missing Value Handling

An assessment of missing values indicated that the *body text* field contained missing entries, while all other variables were complete. To address this, missing text values in both the title and body fields were replaced with empty strings to preserve textual structure for analysis. In addition, rows with missing values in critical modelling variables, including upvotes, timestamp, and subreddit, were identified for removal. However, no such rows were present in the dataset, and therefore no observations were removed. As a result, the dataset size remained unchanged at 29,179 observations.

```
print("Missing values per column:")
print(df.isna().sum())
```

```
Missing values per column:
ID                    0
Title                 0
Body              12774
Upvotes               0
creation_date         0
Subreddit             0
num_comments          0
dtype: int64
```

```
# Replace missing text with empty string
df["Title"] = df["Title"].fillna("")
df["Body"] = df["Body"].fillna("")

# Drop rows missing critical modelling fields
before = df.shape[0]
df = df.dropna(subset=["Upvotes", "creation_date", "Subreddit"])
after = df.shape[0]

print("Rows removed due to missing critical fields:", before - after)
print("Shape after missing value handling:", df.shape)
```

```
Rows removed due to missing critical fields: 0
Shape after missing value handling: (29179, 7)
```

Figure C.3: Handling of Missing Values in the Dataset

Appendix C.4 Duplicate Removal

Duplicate posts were identified using unique Reddit post identifiers to ensure data integrity. A total of 843 duplicate entries were detected and subsequently removed from the dataset. Following this deduplication process, the dataset size was reduced to 28,336 observations. This step ensures that repeated entries do not introduce bias into the modelling process or distort analytical results.

```
# Check duplicates
duplicate_count = df.duplicated(subset=["ID"]).sum()
print("Duplicate rows detected (by ID):", duplicate_count)

# Remove duplicates
before = df.shape[0]
df = df.drop_duplicates(subset=["ID"])
after = df.shape[0]

print("Duplicate rows removed:", before - after)
print("Dataset size after deduplication:", df.shape)
```

```
Duplicate rows detected (by ID): 843
Duplicate rows removed: 843
Dataset size after deduplication: (28336, 7)
```

Figure C.4: Duplicate Record Identification and Removal

Appendix C.5 Data Integrity Checks

Final validation checks were conducted to confirm the consistency and correctness of the dataset after preprocessing. Upvote values were found to range from 0 to 199,296, indicating a wide distribution of engagement levels. The dataset included 30 unique subreddits, reflecting diversity in community sources. In addition, summary statistics were examined to ensure reasonable distributions for both upvotes and comment counts. These checks confirm that the dataset is clean, consistent, and suitable for subsequent feature engineering and modelling.

Integrity Checks

```
print("Upvotes min/max:", df["Upvotes"].min(), df["Upvotes"].max())
print("Number of unique subreddits:", df["Subreddit"].nunique())
print(df[["Upvotes", "num_comments"]].describe())
```

```
Upvotes min/max: 0 199296
Number of unique subreddits: 30
             Upvotes  num_comments
count   28336.000000  28336.000000
mean     3954.029221    165.082157
std     13181.436901    492.276645
min         0.000000      0.000000
25%        14.000000      6.000000
50%       271.000000     41.000000
75%       898.000000    116.000000
max    199296.000000  15433.000000
```

Figure C.5: Data Integrity Checks and Summary Statistics

Appendix D: Text Preprocessing and Feature Preparation

Appendix D.1 Combining Title and Body Text

To create a unified textual representation for analysis, the title and body fields were first standardised by replacing missing values with empty strings and ensuring consistent string formatting. The two fields were then concatenated to form a single text variable (*full_text*), which captures the complete content of each Reddit post. This combined representation allows downstream models to utilise both headline and detailed content simultaneously, improving the richness of textual features used for prediction. As illustrated in **Figure D.1**, the resulting *full_text* column integrates information from both title and body fields.

```
# total number of characters in cleaned text
df["text_length"] = df["text_clean"].str.len()

# total number of words in cleaned text
df["word_count"] = df["text_clean"].str.split().apply(len)

# measure title and body length separately
df["title_length"] = df["Title"].str.len()
df["body_length"] = df["Body"].str.len()

# Quick check of created features
print("Text feature columns created:",
      ["text_length", "word_count", "title_length", "body_length"])

df[["text_length", "word_count", "title_length", "body_length"]].head()
```

```
Text feature columns created: ['text_length', 'word_count', 'title_length', 'body_length']
```

	text_length	word_count	title_length	body_length
0	619	116	23	601
1	841	169	64	789
2	1293	237	101	1289
3	302	65	246	84
4	425	87	26	402

Figure D.1: Extraction of Text-Based Length Features from Reddit Post

Appendix D.2 Text Normalisation

Following text consolidation, a light normalisation process was applied to standardise textual input. All text was converted to lowercase to ensure consistency and reduce redundancy caused by case variations. In addition, extra whitespace and newline characters were removed and replaced with a single space. This step ensures that the text data is in a clean and uniform format, facilitating more effective feature extraction methods such as TF-IDF. As shown in **Figure D.2**, the normalised text (*text_norm*) maintains the original meaning while improving consistency.

```python
def normalise_text(text: str) -> str:
    # convert text to lowercase
    text = text.lower()

    # replace multiple spaces/newlines with a single space
    text = re.sub(r"\s+", " ", text)

    return text.strip()

# create normalised text column
df["text_norm"] = df["full_text"].apply(normalise_text)

# quick check that column was created
print("text_norm created:", "text_norm" in df.columns)
df[["full_text", "text_norm"]].head()
```

```
text_norm created: True
```

	full_text	text_norm
0	Can't have sex the same I'm in a relationship ...	can't have sex the same i'm in a relationship ...
1	How to break up with a friend in the most peac...	how to break up with a friend in the most peac...
2	[MODPOST] [META] Lets bring this subreddit bac...	[modpost] [meta] lets bring this subreddit bac...
3	The most important thing when texting a girl i...	the most important thing when texting a girl i...
4	My dad found my porn, help So.. I got a virus ...	my dad found my porn, help so.. i got a virus ...

Figure D.2: Text Normalisation Process for Reddit Post Content

Appendix D.3 Noise Removal

To further improve data quality, noise removal techniques were applied to the normalised text. This process involved removing URLs, HTML tags, and standalone numerical values that do not contribute meaningful semantic information for modelling. Additional cleaning steps were performed to eliminate extra spaces and ensure well-structured text.

A total of 12,404 rows were identified as containing removable noise elements. The cleaning process produces a refined text variable (text_clean) that is more suitable for machine learning models. As demonstrated in **Figure D.3**, the cleaned text retains the original meaning while removing irrelevant components, resulting in clearer and more informative input for analysis.

```python
def remove_urls_html(text: str) -> str:

    # Safety check for missing values
    if pd.isna(text):
        return ""

    # Remove URLs
    text = re.sub(r'https?://\S+|www\.\S+', ' ', text)

    # Remove HTML tags
    text = re.sub(r'<.*?>', ' ', text)

    # Remove standalone numbers only
    text = re.sub(r"\b\d{1,2}\b", " ", text)  # remove 1-2 digit only

    # Clean extra spaces
    text = re.sub(r'\s+', ' ', text).strip()

    return text

# Apply cleaning to normalised text

df["text_clean"] = df["text_norm"].apply(remove_urls_html)

print("text_clean created:", "text_clean" in df.columns)

# Evidence: show rows that originally contained noise
# (URLs, HTML, or standalone numbers)

noise_mask = df["text_norm"].str.contains(
    r'https?://|www\.|<.*?>|\b\d+\b',
    regex=True,
    na=False
)

print("Number of rows containing removable noise:", noise_mask.sum())

print("\nBefore vs After cleaning examples:")
df.loc[noise_mask, ["text_norm", "text_clean"]].head(10)
```

```
text_clean created: True
Number of rows containing removable noise: 12404

Before vs After cleaning examples:
```

	text_norm	text_clean
0	can't have sex the same i'm in a relationship ...	can't have sex the same i'm in a relationship ...
2	[modpost] [meta] lets bring this subreddit bac...	[modpost] [meta] lets bring this subreddit bac...
3	the most important thing when texting a girl i...	the most important thing when texting a girl i...
5	how to tell my parents i want to get medical a...	how to tell my parents i want to get medical a...
6	golden advice that can help you 1. if you wast...	golden advice that can help you . if you waste...
7	if you need advice i've got some for you if yo...	if you need advice i've got some for you if yo...
8	everything's changing i'm 17 and a senior, so ...	everything's changing i'm and a senior, so you...
9	i'm 14 and i lied about having a gf and a girl...	i'm and i lied about having a gf and a girl wi...
10	advice on crushes? it's pretty basic to have t...	advice on crushes? it's pretty basic to have t...
11	i'm 13 and i still play with toys, and everyon...	i'm and i still play with toys, and everyone (...

Figure D.3: Removal of Noise Elements (URLs, HTML Tags, and Numbers)

Appendix E: Temporal Feature Engineering

To capture time-based patterns in Reddit engagement, temporal features were derived from the post creation timestamp. The creation_date variable was first verified to be in datetime format to ensure accurate feature extraction, and sample timestamps were inspected to confirm correctness.

Based on this timestamp, several temporal features were generated, including the hour of posting (post_hour), day of the week (post_dayofweek), and month of posting (post_month). These features capture when a post was published and allow the model to learn temporal patterns associated with user activity and engagement behaviour. As shown in **Figure E.1**,

the extracted temporal variables provide structured representations of posting time across multiple dimensions.

```
print("creation_date dtype:", df["creation_date"].dtype)

# Display sample timestamps to visually verify correctness
df[["creation_date"]].head()
```

```
creation_date dtype: datetime64[ns]
```

	creation_date
0	2020-03-03 12:39:21
1	2020-03-01 20:44:48
2	2013-01-09 10:23:30
3	2020-03-03 09:33:01
4	2020-02-26 04:51:52

```
# These features capture posting time patterns that may which may influence Reddit engagement and virality.
# Hour of posting (0-23)
df["post_hour"] = df["creation_date"].dt.hour

# Day of week (Monday=0, Sunday=6)
df["post_dayofweek"] = df["creation_date"].dt.dayofweek

# Month of posting (1-12)
df["post_month"] = df["creation_date"].dt.month

print("Temporal features created: post_hour, post_dayofweek, post_month")

# Quick preview
df[["creation_date", "post_hour", "post_dayofweek", "post_month"]].head()
```

```
Temporal features created: post_hour, post_dayofweek, post_month
```

	creation_date	post_hour	post_dayofweek	post_month
0	2020-03-03 12:39:21	12	1	3
1	2020-03-01 20:44:48	20	6	3
2	2013-01-09 10:23:30	10	2	1
3	2020-03-03 09:33:01	9	1	3
4	2020-02-26 04:51:52	4	2	2

Figure E.1: Extraction of Temporal Features from Post Creation Timestamp

In addition to these features, a binary weekend indicator (*is_weekend*) was created to distinguish between weekday and weekend posts. Posts published on Saturday and Sunday were labelled as 1, while weekday posts were labelled as 0, following the standard pandas weekday convention. This feature was introduced based on the assumption that engagement patterns may vary depending on user activity across different days of the week.

Validation checks were performed to ensure that all extracted temporal features were within expected ranges. The *post_hour* values ranged from 0 to 23, *post_dayofweek* values ranged from 0 to 6, and *post_month* values ranged from 1 to 12. In addition, the distribution of the weekend indicator showed that approximately 24.8% of posts occurred on weekends, while 75.2% occurred on weekdays. As illustrated in **Figure E.2**, these results confirm that the

```python
# Weekend posts may exhibit different engagement patterns.
# Saturday = 5, Sunday = 6 (pandas weekday convention)

df["is_weekend"] = df["post_dayofweek"].isin([5, 6]).astype(int)

print("Weekend feature created.")

# Quick check
df[["post_dayofweek", "is_weekend"]].head()
print("\nWeekend distribution:")
print(df["is_weekend"].value_counts(normalize=True))
```

```
Weekend feature created.

Weekend distribution:
is_weekend
0    0.7518
1    0.2482
Name: proportion, dtype: float64
```

```python
# These checks ensure extracted time features are valid
# and free from unexpected values.

print("post_hour range:",
      df["post_hour"].min(), "to", df["post_hour"].max())

print("Unique dayofweek values:",
      sorted(df["post_dayofweek"].unique()))

print("Unique month values:",
      sorted(df["post_month"].unique()))

print("\nWeekend distribution:")
print(df["is_weekend"].value_counts(normalize=True))
```

```
post_hour range: 0 to 23
Unique dayofweek values: [0, 1, 2, 3, 4, 5, 6]
Unique month values: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

Weekend distribution:
is_weekend
0    0.7518
1    0.2482
Name: proportion, dtype: float64
```

Figure E.2: Creation and Validation of Weekend Indicator and Temporal

Appendix D: XGBoost Regressor

An XGBoost Regressor was implemented using the XGBRegressor class from the XGBoost library to evaluate the performance of gradient-boosted regression trees.

The baseline model was trained and used to predict on the test dataset, with regression metrics shown in **Figure D.1**.

```
Baseline XGBoost Regressor Results

MAE:  0.4483
RMSE: 0.6125
R²:   0.9511
```

Figure D.1: Baseline Random Forest Regressor Results

Hyperparameter tuning was performed using GridSearchCV with 5-Fold Cross-Validation, exploring combinations of parameters including the number of estimators, tree depth, and learning rate.

```python
# Base model
xgb_reg_gs = XGBRegressor(
    random_state=42,
    n_jobs=-1,
    tree_method="hist"
)

# Parameter grid
param_grid_xgb_reg = {
    "n_estimators": [200, 400],
    "max_depth": [4, 6],
    "learning_rate": [0.05, 0.1],
    "subsample": [0.8],
    "colsample_bytree": [0.8]
}

# Grid search with k-fold CV
grid_xgb_reg = GridSearchCV(
    estimator=xgb_reg_gs,
    param_grid=param_grid_xgb_reg,
    cv=KFold(n_splits=5, shuffle=True, random_state=42),
    scoring="neg_root_mean_squared_error",
    n_jobs=-1,
    verbose=1
)

# Fit on TRAINING DATA ONLY
grid_xgb_reg.fit(X_train, y_reg_train)
```

Figure 4.33: GridSearchCV Setup for Random Forest Regressor

The optimal parameter configuration identified during the grid search is shown below.

```
Best XGB Regressor parameters: {'colsample_bytree': 0.8, 'learning_rate': 0.05, 'max_depth': 6, 'n_estimators': 400, 'subsample': 0.8}
Best XGB Regressor CV score (neg RMSE): -0.6300341728544969
Best XGB Regressor CV RMSE: 0.6300341728544969
```

Figure 4.34: Optimal Hyperparameters (Random Forest Regressor)

The tuned model was subsequently evaluated on the test dataset using the selected parameters.

```
Tuned XGBoost Regressor Results

MAE:  0.4453
RMSE: 0.6092
R²:   0.9517
```

Figure 4.35: Tuned XGBoost Regressor Results

The results show that the XGBoost Regressor achieved strong predictive performance and performed comparably to the Random Forest Regressor when predicting log-transformed Reddit upvote counts.

Appendix G: Cross-Validation Stability

To further assess model robustness, Stratified 5-Fold Cross-Validation was applied to the training dataset.

	Metric	Mean	Std
0	Accuracy	0.9445	0.0027
1	Precision	0.6841	0.0146
2	Recall	0.8289	0.0147
3	F1	0.7494	0.0108
4	ROC-AUC	0.9648	0.0045

Figure G.1: Cross-Validation Performance Metrics

The cross-validation results show that the Logistic Regression model achieved a mean accuracy of 0.9445 and a mean F1-score of approximately 0.749, with relatively low variation across validation folds. This indicates that the model performance remains stable when trained on different subsets of the data.

The consistency observed in the cross-validation results suggests that the classification models demonstrate good generalisation capability and are unlikely to be overfitting to the training dataset, meaning that similar performance can be expected when applied to unseen Reddit posts.

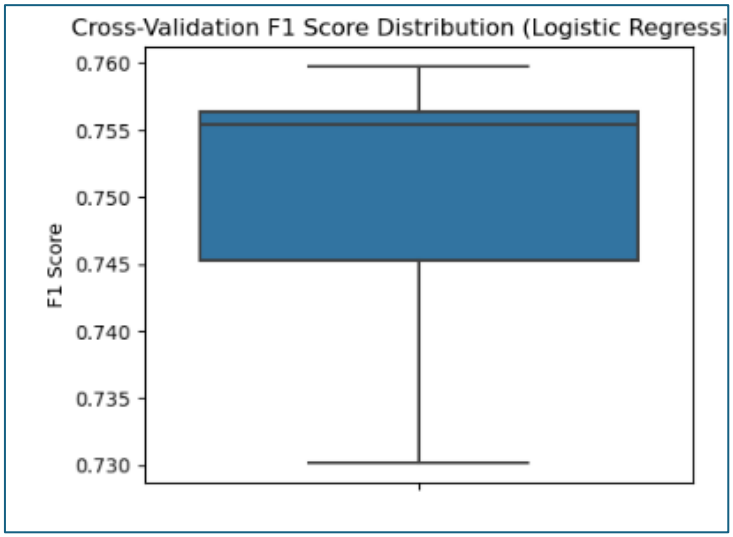


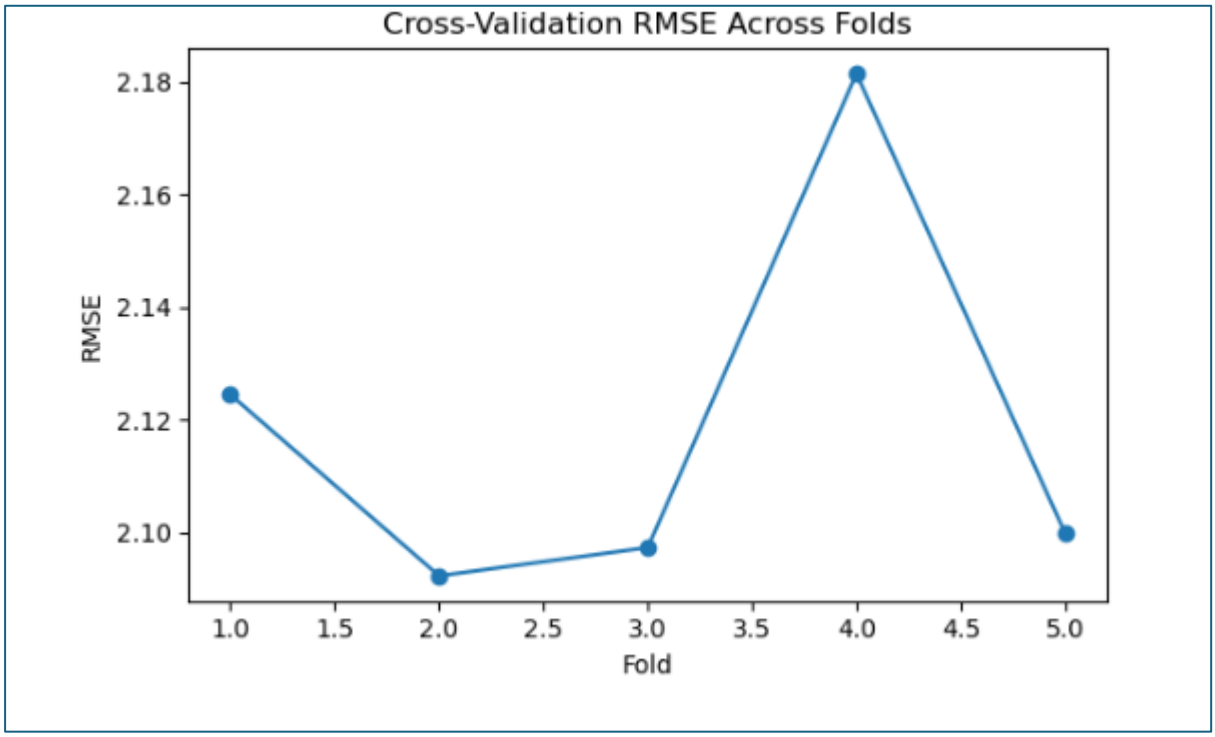
Figure G.2: Distribution of F1 Scores Across Stratified 5-Fold Cross-Validation

Appendix H: Additional Regression Model Diagnostic Analysis

Additional diagnostic analyses were conducted to further examine the stability and prediction behaviour of the regression models used for Reddit upvote prediction.

Appendix H.1 Cross-Validation RMSE Across Folds (Linear Regression)

To further assess model stability, 5-fold cross-validation was applied to the Linear Regression model. The RMSE values obtained across validation folds are shown in **Figure H.1**.



Appendix H.1 Cross-Validation RMSE Across Folds (Linear Regression)

The RMSE values remain relatively consistent across folds, ranging between approximately 2.09 and 2.18. Although these errors are relatively high compared with the ensemble models, the limited variation across folds indicates that the model performance remains stable when trained on different subsets of the data.

This suggests that the Linear Regression model demonstrates reasonable generalisation capability despite its lower predictive accuracy. However, the consistently high RMSE values reinforce the earlier observation that linear modelling assumptions are insufficient for capturing the complex factors influencing Reddit engagement.

Appendix H.2 Residual Distribution Comparison

The residual distribution plots shown in **Figure H.2** further highlight the differences in model performance.

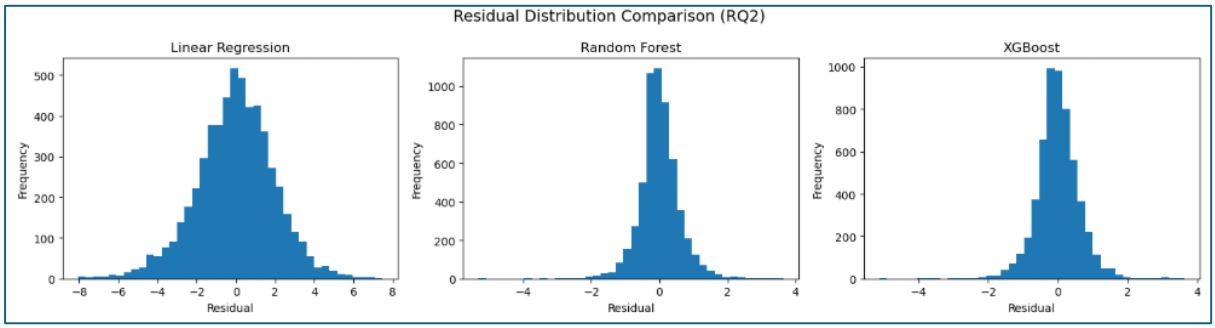


Figure H.2: Comparison of Residual Distributions for Regression Models

The Linear Regression residuals display a wider distribution, indicating larger and more variable prediction errors. In contrast, the residual distributions for Random Forest and XGBoost are more concentrated around zero, suggesting smaller and more consistent prediction errors.

This indicates that the ensemble models provide a closer approximation of the true engagement values and achieve improved predictive accuracy compared with the linear baseline.

Appendix H.3 Prediction Error vs Actual Values

The prediction error vs actual value plots shown in **Figure H.3** illustrate how prediction errors vary across different engagement levels.

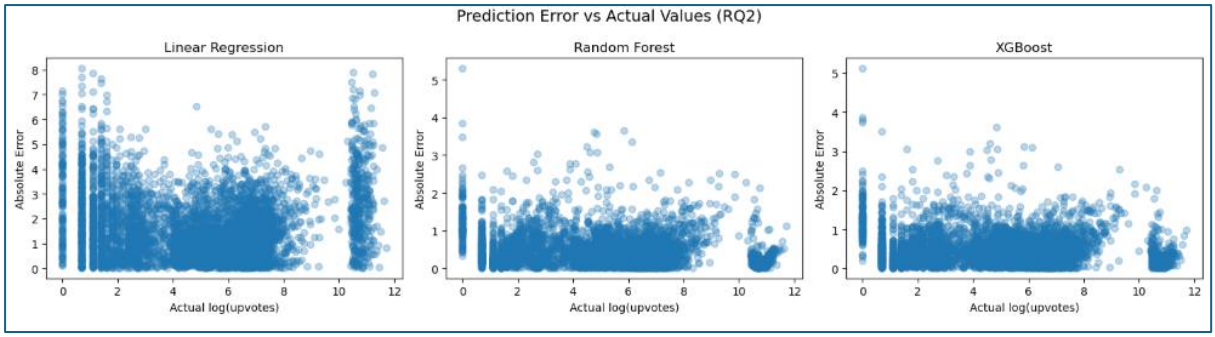


Figure H.3: Relationship Between Prediction Error and Actual Engagement

For the Linear Regression model, prediction errors remain large across a wide range of engagement values, particularly for posts with higher upvotes. This indicates that the model struggles to accurately predict highly engaged posts.

In contrast, the Random Forest and XGBoost models exhibit smaller and more stable prediction errors across most engagement levels. However, slightly larger errors are still observed for posts with extremely high engagement values. This suggests that although the ensemble models provide strong predictive performance, predicting very high engagement remains challenging due to the inherent variability and unpredictability of viral online content.

Overall, these results demonstrate that ensemble-based regression models provide substantially more accurate predictions of Reddit engagement compared with traditional linear regression approaches.

Appendix I: Additional Time-Series Diagnostic Analysis

This appendix presents additional diagnostic analyses conducted to further evaluate the performance of the ARIMA model used in the time-series analysis of Reddit engagement (RQ3). These diagnostics examine forecast errors and residual autocorrelation to assess whether the model adequately captures the temporal structure of the data.

Appendix I.1 Forecast Error Analysis

To further evaluate model performance, the forecast errors generated during the evaluation period were analysed.

Forecast Errors Over Time

Figure I.1 shows the forecast errors across the evaluation period. The errors fluctuate around the zero baseline, indicating that the ARIMA model does not consistently overestimate or underestimate Reddit engagement. However, several larger deviations are visible during weeks with substantial engagement changes. This suggests that while the model captures the overall engagement trend, predicting sudden spikes remains difficult due to the highly volatile nature of social media activity.

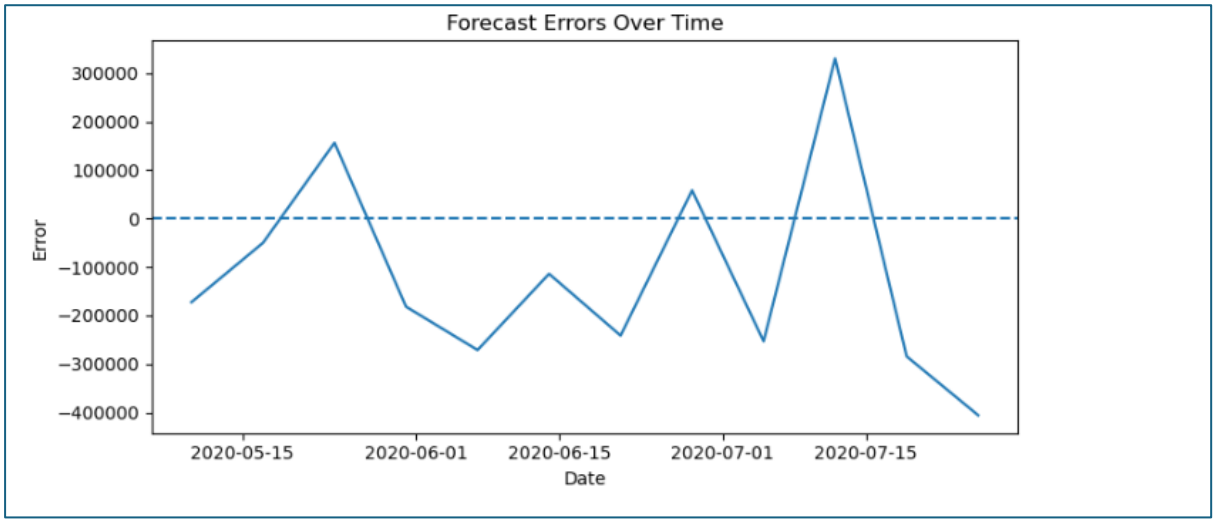


Figure I.1: Temporal Variation of Forecast Errors in Time-Series Prediction

Forecast Error Distribution

Figure I.2 illustrates the distribution of forecast errors produced by the ARIMA model. The errors are distributed across both positive and negative values, indicating that the model occasionally overestimates and underestimates engagement levels. The spread of errors reflects the variability of Reddit engagement and highlights the difficulty of forecasting highly dynamic online activity.

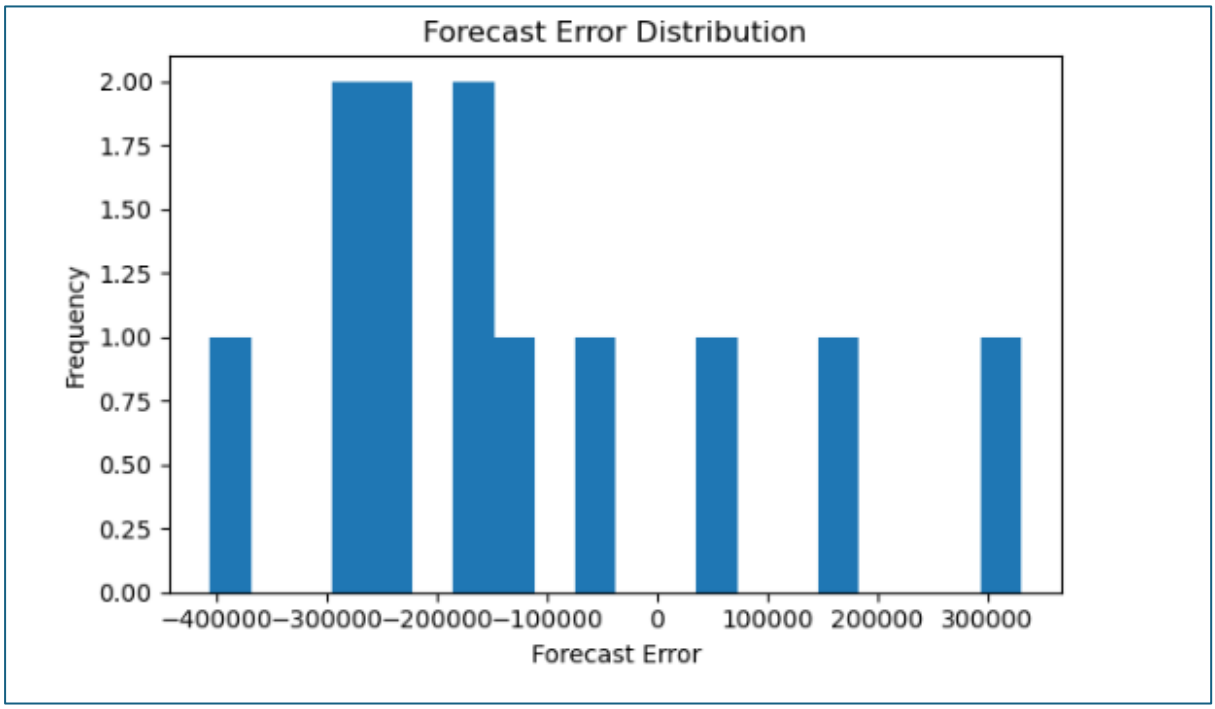


Figure I.2: Frequency Distribution of Forecast Errors

Appendix I.2 Residual Autocorrelation Diagnostics

To assess whether the ARIMA model adequately captured the temporal structure of the data, residual autocorrelation diagnostics were conducted using Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) plots.

Autocorrelation Function (ACF)

Figure I.2 presents the autocorrelation function of the forecast residuals. Most autocorrelation values fall within the confidence bounds, indicating that there is little remaining temporal dependence in the residual errors. This suggests that the ARIMA model has successfully captured much of the temporal structure present in the engagement data.

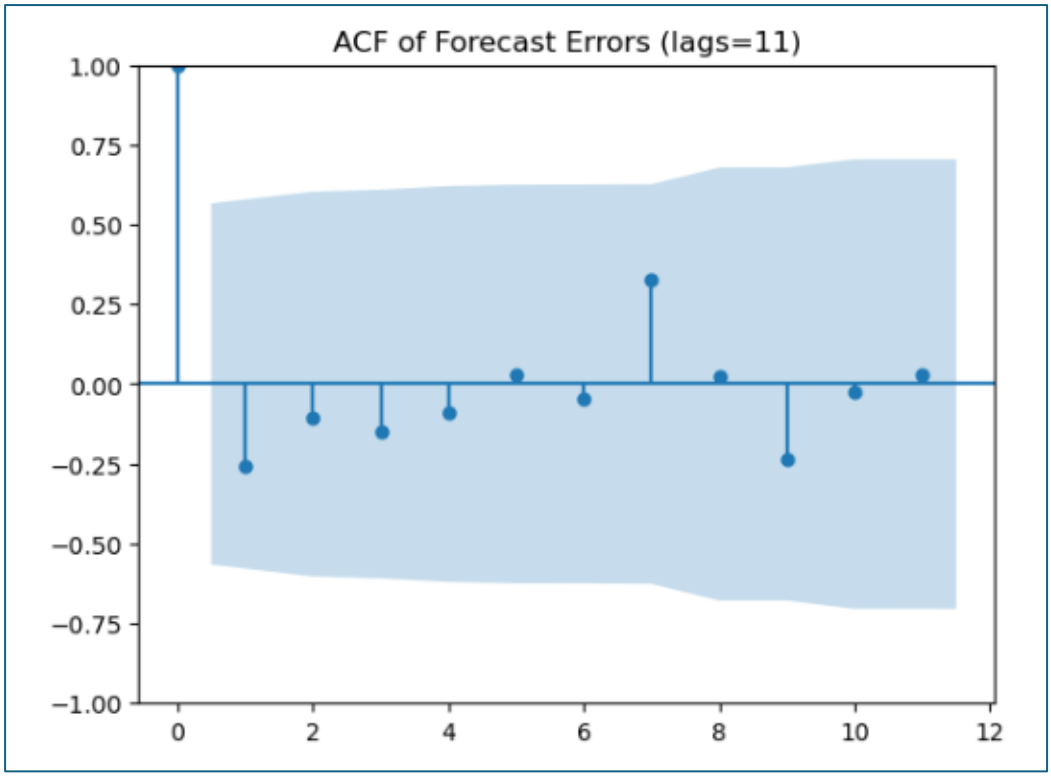


Figure I.3: Autocorrelation Function (PACF) of Forecast Errors

Partial Autocorrelation Function (PACF)

Figure I.3 shows the partial autocorrelation function of the residual errors. Similar to the ACF results, most values remain within the confidence bounds, indicating that little additional structure remains in the residuals. This suggests that the residual errors behave approximately like random noise, supporting the adequacy of the ARIMA model specification.

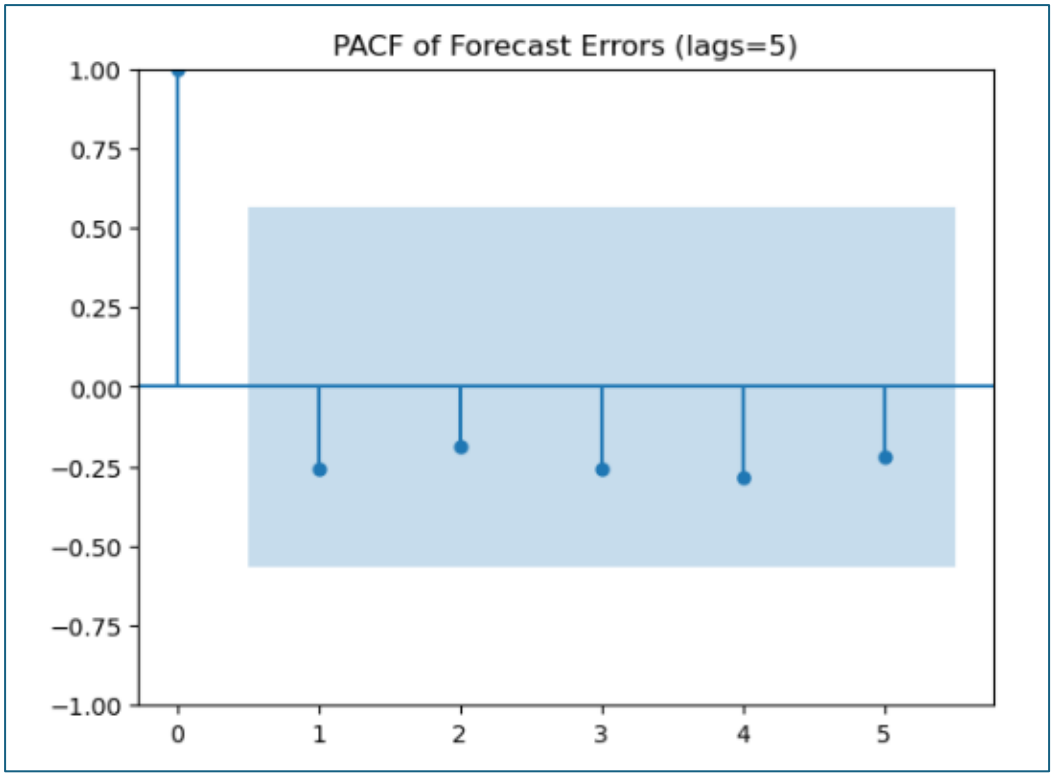


Figure H.X: Partial Autocorrelation Function (PACF) of Forecast Errors

Appendix J: Additional Network Analysis Results

Appendix J.1 Correlation Analysis of Network Metrics

To further examine the relationship between subreddit network position and viral engagement, additional correlation analyses were conducted using both Pearson and Spearman correlation coefficients. These tests evaluate whether subreddit viral rates are associated with structural network metrics, including degree centrality, betweenness centrality, and weighted node strength.

The Pearson correlation results indicate that the relationship between viral rate and the network centrality measures is extremely weak. For example, the correlation between viral rate and degree centrality is approximately −0.01, while the correlation between viral rate and betweenness centrality is approximately 0.03. These values are close to zero, suggesting that a subreddit's structural influence within the network does not strongly relate to its likelihood of producing viral posts.

To verify that the results are not dependent on linear assumptions, a Spearman rank correlation analysis was also conducted. The Spearman results show similarly small relationships between viral rate and the centrality metrics, further supporting the conclusion that network position has minimal influence on viral engagement.

Pearson Correlation Matrix:

	viral_rate	degree_centrality	betweenness_centrality	weighted_strength
viral_rate	1.000000	-0.007295	0.027462	-0.001980
degree_centrality	-0.007295	1.000000	0.528203	0.911955
betweenness_centrality	0.027462	0.528203	1.000000	0.378754
weighted_strength	-0.001980	0.911955	0.378754	1.000000

Spearman Correlation Matrix:

	viral_rate	degree_centrality	betweenness_centrality	weighted_strength
viral_rate	1.000000	-0.065480	0.042356	-0.090613
degree_centrality	-0.065480	1.000000	0.822187	0.806261
betweenness_centrality	0.042356	0.822187	1.000000	0.515512
weighted_strength	-0.090613	0.806261	0.515512	1.000000

Figure J.1: Pearson and Spearman Correlation Matrices of Network Centrality Metrics and Viral Rate

Appendix J.2 Correlation Heatmap

To provide a visual summary of the relationships between network metrics and viral engagement, a correlation heatmap was generated. The heatmap illustrates the strength of the relationships between viral rate and the centrality measures.

As shown in **Figure J.2**, the correlations between viral rate and the network centrality metrics are close to zero. This visual representation further confirms that network position does not strongly influence viral engagement across the analysed Reddit communities.

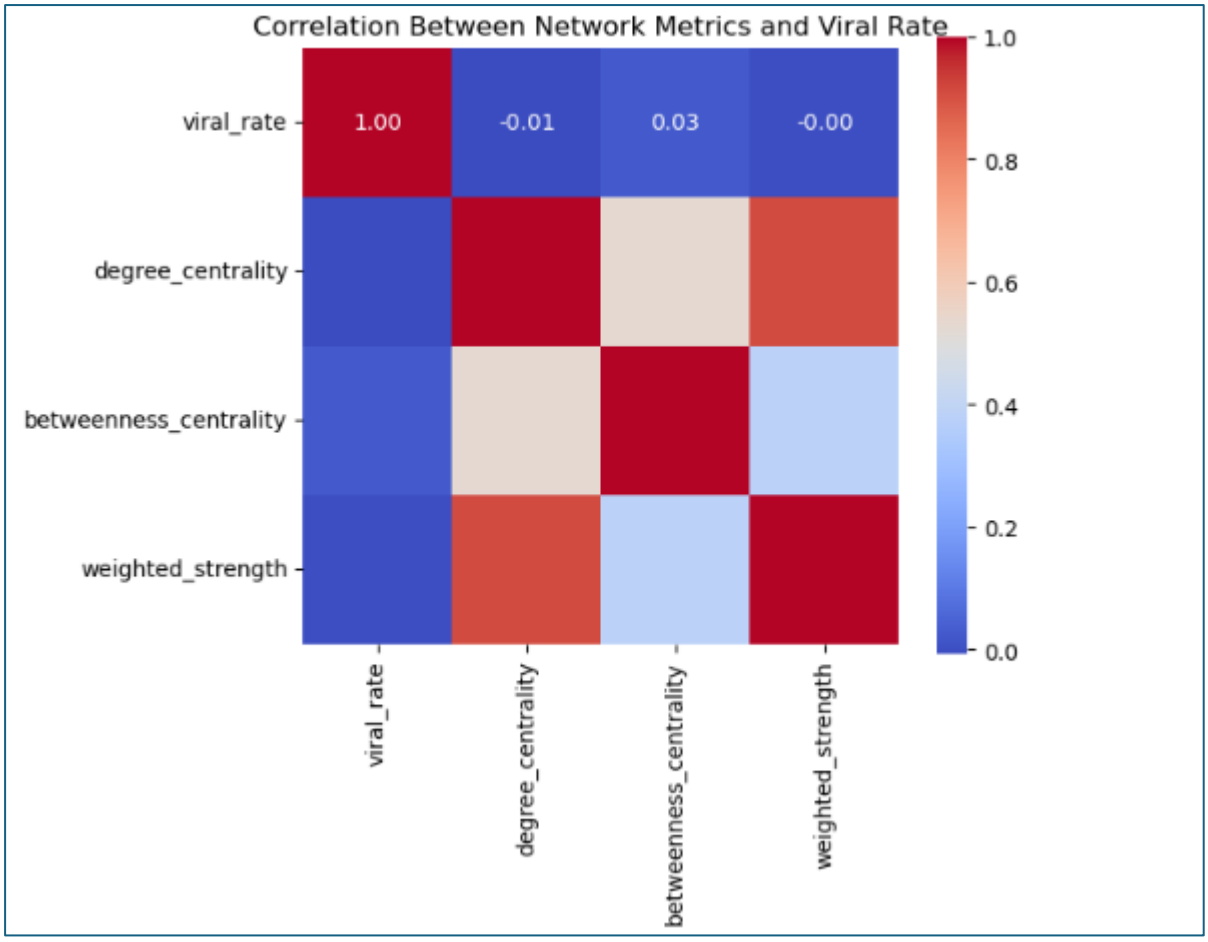


Figure J.2: Correlation Heatmap of Network Centrality Metrics and Viral Rate

Appendix J.3 Subreddit Activity as a Potential Confounding Factor

An additional analysis was conducted to examine whether subreddit activity levels could influence the observed results. Specifically, the number of posts contributed by each subreddit (post_count) was analysed to determine whether larger or more active communities tend to produce higher viral rates or have higher centrality values.

Spearman correlation tests were performed between post count and viral rate, as well as between post count and degree centrality. The results indicate weak and statistically insignificant relationships, suggesting that subreddit size does not substantially bias the relationship between network centrality and viral engagement.

```python
# Check whether subreddit activity (post_count) influences
# viral rate or network centrality
# This helps determine if larger subreddits bias the results

from scipy.stats import spearmanr

# Spearman correlation: subreddit size vs viral rate
rho_pc_v, p_pc_v = spearmanr(eval_df["post_count"], eval_df["viral_rate"])

# Spearman correlation: subreddit size vs degree centrality
rho_pc_d, p_pc_d = spearmanr(eval_df["post_count"], eval_df["degree_centrality"])

print(f"Spearman (post_count vs viral_rate): rho={rho_pc_v:.3f}, p={p_pc_v:.4f}")
print(f"Spearman (post_count vs degree_centrality): rho={rho_pc_d:.3f}, p={p_pc_d:.4f}")
```

```
Spearman (post_count vs viral_rate): rho=0.168, p=0.3759
Spearman (post_count vs degree_centrality): rho=0.046, p=0.8107
```

Figure J.3: Spearman Correlation Between Post Count, Viral Rate, and Degree

8 REFERENCES

- Alonso-López, N., Redondo, T., & Revilla, M. (2020).
  What makes a post successful on Instagram? Analyzing the relationship between metadata and engagement. International Journal of Interactive Multimedia and Artificial Intelligence, 6(2), 13–22.
- Kim, Y., & Hwang, J. (2025).
  Predicting social media engagement from emotional and temporal features. arXiv preprint arXiv:2508.21650.
- Makow, N., Millican, M., & Yu, C. (2021).
  A network model for Reddit post virality prediction. Stanford University.
- Pinto, H., Almeida, J. M., & Gonçalves, M. A. (2013).
  Using early view patterns to predict the popularity of YouTube videos. In Proceedings of the 6th ACM International Conference on Web Search and Data Mining (pp. 365–374).
- Roy, A., & Basu, S. (2020).
  Emotional dynamics and the spread of political messages on Twitter. Social Network Analysis and Mining, 10(1), 1–14.
- Zhang, H., Li, H., & Li, X. (2022).
  Slapping cats, bopping heads, and Oreo shakes: Understanding indicators of virality in TikTok short videos. arXiv preprint.
- Freeman, L. C. (1979).
  Centrality in social networks: Conceptual clarification. Social Networks, 1(3), 215–239.
- Han, J., Pei, J., & Kamber, M. (2012). Data mining: Concepts and techniques (3rd ed.). Elsevier.

- Molnar, C. (2022). Interpretable machine learning. Christoph Molnar.
- Kumar, S., Hamilton, W., Leskovec, J., & Jurafsky, D. (2020). Community interaction and engagement dynamics in online social networks. Proceedings of the International AAAI Conference on Web and Social Media.
- Keim, D., Mansmann, F., Schneidewind, J., Thomas, J., & Ziegler, H. (2010). Visual analytics: Scope and challenges. In Visual Data Mining (pp. 76–90). Springer.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., & Willing, C. (2016). Jupyter notebooks – a publishing format for reproducible computational workflows. In F. Loizides & B. Schmidt (Eds.), Positioning and power in academic publishing: Players, agents and agendas (pp. 87–90). IOS Press.
- PrakharRathi25. (n.d.). Reddit data (huge) [Dataset]. Kaggle. https://www.kaggle.com/datasets/prakharrathi25/reddit-data-huge/data